Atlas Orbitalis 用户手册

Atlas Orbitalis（简称 Atlas）是一套面向航天器轨道计算与分析的综合 Fortran 工具库，由南京大学 Shawn Hu 开发，基于 MIT 许可证发布。

本手册面向库的使用者，介绍各模块的功能、关键数据结构、使用方法和基础理论。手册共分为 15 章并附一个附录，各章相对独立，读者可按需查阅。

目录

章节	简介
第1章 简介	项目概述、核心功能、设计理念、单位制、坐标系约定、模块分层架构
第2章 构建与安装	编译器要求、Makefile 使用、依赖库、运行测试、项目文件结构
第3章 时间系统	时间表示 `type_t`、MJD 儒略日、时间尺度（TT, TAI, UTC, UT1, GPS）、地球定向参数
第4章 坐标系统	参考框架（GCRS, ITRS 等）、坐标变换、SOFA 接口、`type_cartesian`
第5章 轨道根数	四类轨道根数（Keplerian, Cartesian, NSG1, NSG2）、根数转换、非奇异根数
第6章 力模型	地球非球形引力、大气阻力、太阳光压、第三体引力、后牛顿效应、推力模型
第7章 轨道预报	数值积分（RK8）、力模型开关配置、轨迹生成与输出
第8章 卫星对象	`type_satellite` 创建与初始化、状态管理、轨迹保存、多星场景
第9章 地面站	`type_station` 地面站定义、大地坐标转换、J2000 轨迹生成
第10章 可见性与观测类型	观测类型（赤经赤纬/方位仰角/测距/多普勒）、观测几何、可见性约束、对流层折射
第11章 轨道确定	观测残差与法方程、初轨确定 IOD、精密定轨 POD、批处理最小二乘、EKF 卡尔曼滤波

章节	简介
第12章 三体问题	圆型限制性三体问题、Halo 轨道、Lissajous 轨道、Lambert 问题
第13章 行星历表	JPL DE440 历表接口、行星/日月位置速度获取
第14章 姿态模型	航天器姿态描述、指向模式、姿态变换
第15章 应用程序	15 个命令行程序介绍与参数说明
附录 模块参考	完整模块索引、关键类型速查表、常数与单位转换、文件格式

阅读建议

- **基础入门**：建议依次阅读第1章（简介）、第2章（构建）、第3章（时间）、第4章（坐标）、第5章（轨道根数），建立基础概念
- **轨道预报**：重点阅读第5–8章，了解轨道根数、力模型、数值积分和卫星对象的使用
- **轨道确定**：重点阅读第3、9、10、11章，了解时间系统、地面站、观测模型和定轨算法
- **深空探测**：重点阅读第12章（三体问题）和第13章（行星历表）
- **日常参考**：查阅附录中的模块索引和常数速查表

约定

- 手册使用中文撰写，代码示例使用 Fortran 语法
- 引用模块时使用 `` `module_xxx` `` 格式
- 所有内部计算采用**归一化单位制**（长度单位 = 1 地球赤道半径，时间单位 ≈ 806.81 秒，质量单位 = 1 地球质量），输入输出在边界处转换为 SI 单位
  \newpage

第1章 简介

1.1 项目概述

Atlas Orbitalis（简称 Atlas）是一套面向航天器轨道计算与分析的综合 Fortran 工具库。项目名称结合了希腊神话中的擎天神 Atlas（肩负天球）与拉丁语 Orbitalis（轨道），寓意支撑轨道力学计算的基础工具。

Atlas 提供卫星轨道力学领域的主要功能：

- **轨道预报**：数值方法与分析方法的轨道外推
- **观测仿真**：生成地面站对卫星的仿真跟踪数据
- **初轨确定（IOD）**：从有限观测数据确定初步轨道
- **精密定轨（POD）**：基于大量跟踪数据的高精度轨道计算
- **卡尔曼滤波**：实时轨道估计与预测
- **多体动力学**：圆形限制性三体问题（CRTBP）、Halo 与 Lissajous 轨道设计

1.2 设计理念

1.2.1 模块化分层架构

Atlas 的源码模块遵循严格的 **7 级分层依赖**。每一级模块只能 `use` 更低级别的模块，形成清晰的单向依赖链：

层级	模块角色	典型模块
Level 0	基础设施	`module_const`，`module_MathsMatrix`，`module_elements`
Level 1	基础动力学与时间	`module_time`，`module_CRTBP`，`module_lambert`
Level 2	行星历表与特殊轨道	`module_planets`，`module_halo`，`module_LissajousOrbit`
Level	坐标与姿态	`module_space`，`module_cartesian`，`module_attitude`

层级	模块角色	典型模块
3		
Level 4	力模型与积分器	`module_Gravity`，`module_atmosphere`，`module_orbitPredictionNumerical`
Level 5	应用层对象	`module_satellite`，`module_station`，`module_ODor`
Level 6	对外接口	`module_Atlas`（重导出 Level 5 模块）

1.2.2 归一化单位制

Atlas 所有内部计算均采用**以地球参数为基准的归一化单位制**，以减少数值误差、改善矩阵条件数：

物理量	归一化单位	SI 换算值
长度	1 地球赤道半径	6378.1363 km
时间	1 时间单位	≈ 806.81 秒
质量	1 地球质量	≈ 5.97×10^{24} kg

相关转换因子（`u_km`，`u_sec`，`u_day` 等）定义在 `module_const` 中。文件 I/O 在边界处完成 SI ↔ 归一化单位的转换——用户输入输出始终使用 SI 单位（km, km/s, 度），无需手动转换。

1.2.3 面向对象的 Fortran 风格

Atlas 充分利用 Fortran 2003+ 的 derived type 和 type-bound procedure 特性，以面向对象的风格组织代码。核心领域对象包括：

- `type_t`：时间表示（MJD 日数 + 当日秒数）
- `type_satellite`：卫星对象，封装状态向量、力模型配置、轨迹数据
- `type_station`：地面站对象
- `type_ODor`：轨道确定引擎（批处理最小二乘 / EKF / 序贯滤波）
- `type_CRTBP`：三体系统对象

1.3 坐标系约定

- **ICRS / J2000.0**：天球参考系，惯性计算的基本框架
- **ITRS / ECF**：地固坐标系，用于地面站位置和地固系输出
- **旋转坐标系**：CRTBP 中使用的随主天体旋转的坐标系

坐标变换通过 `module_space` 和 `module_cartesian` 实现，底层使用项目内置的 IAU SOFA 源码（`sofa/`）。

1.4 轨道根数类型

Atlas 支持四类轨道根数，以整数编码区分：

编码	类型	说明
1	Keplerian	经典开普勒根数 (a, e, i, Ω, ω, M)
2	Cartesian	笛卡尔位置速度 (x, y, z, vx, vy, vz)
3	NSG1	第一类非奇异根数
4	NSG2	第二类非奇异根数（春分点根数）

其中 NSG1 和 NSG2 为非奇异根数，避免了小偏心率和小倾角情况下的奇异性问题，推荐在精密定轨和状态转移矩阵计算中使用。

1.5 力模型系统

Atlas 的力模型通过 8 维开关数组 `K(1:8)` 控制，用户可在运行时灵活组合：

索引	力模型	控制含义
K(1)	Tesseral Harmonics	田谐项（非球形引力高阶摄动）
K(2)	Lunar Perturbation	月球引力摄动
K(3)	Lunar Tide	月球潮汐形变摄动

索引	力模型	控制含义
K(4)	Solar Perturbation	太阳引力摄动
K(5)	Solar Tide	太阳潮汐形变摄动
K(6)	Solar Radiation Pressure	太阳光压摄动
K(7)	Atmospheric Drag	大气阻力摄动
K(8)	Post-Newtonian Effect	后牛顿效应（广义相对论修正）

各力模型的详细原理和参数配置见第6章 力模型。

1.6 依赖与第三方组件

- **JPL DE440 行星历表**：提供太阳、月球和行星的高精度位置。运行时从 `data/data_op/` 加载
- **IAU SOFA**（`sofa/` 目录）：IAU 基础天文学标准库，用于高精度坐标变换和时间转换。SOFA 以**源码**形式随项目提供，构建时与 Atlas 源码一起编译，无需预先安装或链接外部 SOFA 库
- **NRLMSISE-00 大气模型**：地球高层大气密度计算（以 legacy Fortran .for 文件提供）

1.7 项目结构

```
Atlas/
├── src/          # 源码模块（~40 个 .f90 + 3 个 .for 遗留文件）
├── app/          # 应用程序（15 个独立的命令行程序）
├── test/         # 测试程序（16 个独立测试）
├── data/
│   ├── data_op/  # 运行数据：EOP 表、重力场系数、JPL DE440 历表
│   ├── input/    # 场景配置和输入文件
│   ├── output/   # 输出：轨迹、观测、滤波结果
│   └── doc/      # 专题技术文档
├── sofa/         # IAU SOFA 源码（249 个 .for 文件，随构建一起编译）
├── build/        # 编译产物（.o, .mod, libatlas.a）
├── bin/          # 编译后的可执行文件
├── Makefile      # 主构建系统（gfortran）
└── Atlas.sln     # Visual Studio / Intel Fortran 方案
```

参考文献

1. Montenbruck, O., & Gill, E. (2000). *Satellite Orbits: Models, Methods, and Applications*. Springer.
2. Vallado, D. A. (2013). *Fundamentals of Astrodynamics and Applications* (4th ed.). Microcosm Press.
3. IAU SOFA Board. *IAU SOFA Software Collection*. http://www.iausofa.org
4. JPL/NASA. *DE440 Planetary Ephemeris*.
5. Picone, J. M., et al. (2002). NRLMSISE-00 empirical model of the atmosphere. *JGR*, 107(A12).

\newpage

第2章 构建与安装

2.1 编译器要求

Atlas 支持两种编译器工具链：

工具链	编译器	构建系统	平台
GNU Fortran	`gfortran`	Makefile	Linux / macOS / Windows (MinGW/Cygwin)
Intel Fortran	`ifx`	Visual Studio + `Atlas.sln`	Windows

推荐使用 gfortran + Makefile 方式，跨平台且最为简洁。

最低编译器版本

- gfortran 9.0+（支持 Fortran 2003 的 derived type 与 type-bound procedure）
- Intel Fortran 2021+（ifx）或 19.0+（ifort）

2.2 Makefile 构建

2.2.1 完整构建

```
# 构建全部：库、应用程序、测试
make

# 输出：
#   build/libatlas.a    — Atlas 静态库
#   build/libsofa.a     — SOFA 静态库（由 sofa/ 源码编译）
#   build/*.mod         — 模块接口文件
#   bin/app_*           — 应用程序可执行文件
#   bin/test_*          — 测试程序可执行文件
```

2.2.2 单独构建

```
# 先构建库
make build/libatlas.a

# 再构建单个程序
make bin/test_CRTBP
make bin/app_orbitPrediction
```

2.2.3 清理

```
make clean    # 删除 build/ 和 bin/ 目录
```

2.2.4 编译选项

Makefile 中的默认编译选项为：

```
FC = gfortran
FFLAGS = -O2 -std=gnu -fall-intrinsics -fPIC
```

选项说明：

- `-O2`：二级优化（平衡编译速度和运行性能）
- `-std=gnu`：允许 GNU 扩展（兼容 legacy `.for` 文件）
- `-fall-intrinsics`：启用所有内建函数
- `-fPIC`：生成位置无关代码（用于创建静态库）

如需调试，可修改 `FFLAGS` 为 `-g -Og -std=gnu -fall-intrinsics -fPIC`。

2.3 依赖与第三方组件

2.3.1 SOFA

SOFA（Standards of Fundamental Astronomy）是 IAU 提供的天文学基础标准库，用于高精度坐标变换和时间转换。

Atlas **不使用预编译的 SOFA 库**，而是把 SOFA 的全部 Fortran 源码（249 个 `.for` 文件）直接放在 `sofa/` 目录下，构建时编译为 `build/libsofa.a`，再由所有目标（库、应用程序、测试）链接。因此：

2.3.2 JPL DE440 行星历表

JPL DE440 历表提供太阳、月球和行星的高精度位置速度数据。历表文件应放置在 `data/data_op/` 目录下。

文件格式：JPL 二进制历表（`PLEPH.for` 读取）
路径配置：在 `module_planets` 中设置，不同操作系统有不同默认路径

2.3.3 NRLMSISE-00 大气模型数据

大气模型所需的数据文件已在源代码（`nrlmsise00_sub.for`）中内嵌，无需额外下载。

2.4 运行测试

Atlas 没有使用单元测试框架——每个测试都是独立的 Fortran 程序，输出结果供人工检查。

```
# 构建所有测试
make

# 运行单个测试
./bin/test_CRTBP            # 三体系统计算
./bin/test_JPLEPH            # JPL 历表读取
./bin/test_EarthGravity      # 地球非球形引力
./bin/test_LissajousOrbit    # Lissajous 轨道计算
./bin/test_c2t06a            # IAU2006 坐标变换（SOFA）
./bin/test_sp3               # SP3 轨道文件读取
```

主要测试列表：

测试程序	测试内容
`test_CRTBP`	地月/日地 CRTBP 系统、拉格朗日点计算

测试程序	测试内容
`test_JPLEPH`	JPL DE440 历表月球位置读取
`test_EarthGravity`	GGM03C 非球形引力加速度
`test_LissajousOrbit`	Lissajous 轨道初始化与状态计算
`test_c2t06a`	IAU2006 CIO-based 天球→地固变换
`test_OS`	运行环境检测
`test_lunar_gravity`	月球引力摄动
`test_moon_coordinate_transformation`	月心坐标变换
`test_sp3`	SP3 轨道文件解析
`test_DXDR_nsg2`	NSG2 根数偏导数计算
`test_verify_DXDR_nsg2`	NSG2 根数偏导数验证
`orbit_prediction_ECOM`	ECOM 9 参数光压模型轨道预报
`main_nsg2`	NSG2 根数计算

2.5 Visual Studio 构建（Windows）

对于偏好 Windows IDE 的用户，提供了 `Atlas.sln` 方案文件：

1. 安装 Intel Fortran 编译器（Intel oneAPI HPC Toolkit）
2. 安装 Visual Studio 2019 或更新版本
3. 打开 `Atlas.sln`
4. 选择 Release/Debug 配置
5. 生成解决方案

编译产物输出到 `x64/Release/` 或 `x64/Debug/`。

2.6 项目文件结构

```
Atlas/
├── src/                    # 源码（~40 个 .f90 模块 + 3 个 .for 遗留文件）
│   ├── module_const.f90    # Level 0: 物理常数与单位制
│   ├── module_time.f90     # Level 1: 时间系统
│   ├── module_space.f90    # Level 3: 坐标变换
│   ├── module_satellite.f90 # Level 5: 卫星对象
│   ├── module_ODor.f90     # Level 5: 精密定轨引擎
│   ├── module_Atlas.f90    # Level 6: 对外接口（重导出模块）
│   ├── PLEPH.for           # Legacy: JPL 历表读取
│   └── ...
├── app/                    # 15 个命令行应用程序
├── test/                   # 16 个测试程序
├── data/
│   ├── data_op/            # 运行所需数据文件
│   │   ├── GGM03C.txt      # 地球重力场系数（360阶）
│   │   ├── Nutation.dat    # 章动系数
│   │   └── ...             # JPL DE440 历表文件
│   ├── input/              # 应用程序输入场景
│   ├── output/             # 应用程序输出结果
│   └── doc/                # 专题技术文档
├── sofa/                   # IAU SOFA 源码（编译为 build/libsofa.a，无需外部依赖）
├── build/                  # 编译中间产物 (.o, .mod, libatlas.a)
├── bin/                    # 编译完成的可执行文件
├── Makefile                # 主构建脚本
└── Atlas.sln               # Visual Studio 方案
```

2.7 模块编译顺序

由于模块间存在严格的层级依赖关系，Makefile 定义了 7 个编译层级。如需手动编译，必须遵循以下顺序：

层级	先决条件	模块
Level 0	(无)	const, toolKits, ErrorHandler, progressBar, rotator, randum, String, FileDef, elements, MathsMatrix, SP3Kit

层级	先决条件	模块
Level 1	Level 0	CRTBP, lambert, time, EOPParam
Level 2	Level 0–1	matrix, planets, halo, LissajousOrbit, timeSeries
Level 3	Level 0–2	space, SLE, attitude, cartesian
Level 4	Level 0–3	Gravity, atmosphere, perturbation*, thruster, orbitPredictionNumerical
Level 5	Level 0–4	station, satellite, access, residual, satelliteScenario, IODor, ODor
Level 6	Level 0–5	Atlas

Legacy `.for` 文件（`PLEPH.for`，`nrlmsise00_driver.for`，`nrlmsise00_sub.for`）无模块依赖，可在任意阶段编译。
\newpage

第3章 时间系统

3.1 概述

时间系统是轨道力学计算的基础。Atlas 通过 `module_time` 提供了一套面向对象的时间处理框架，支持多种时间系统的表示与相互转换。地球定向参数（EOP）由 `module_EOPParam` 管理，为坐标变换提供极移和 UT1-UTC 数据。

对应模块：`module_time`（Level 1），`module_EOPParam`（Level 1）

3.2 时间对象：`type_t`

`type_t` 是 Atlas 中表示时刻的基本数据结构。内部以**约化儒略日（MJD, Modified Julian Date）**为核心，分裂为整数日数和当日秒数两部分存储：

```
type type_t
    integer :: ttype    ! 时间系统类型标识
    real(8) :: day      ! MJD 整数日数（JD - 2400000.5）
    real(8) :: sec      ! 当日秒数 [0, 86400)
    real(8) :: val      ! 完整 MJD 值 = day + sec/86400
contains
    procedure :: init    ! 初始化
    procedure :: toDate  ! 格式化为日期字符串
    procedure :: toUTC   ! 转换到 UTC
    procedure :: toTT    ! 转换到 TT
    procedure :: toUT1   ! 转换到 UT1
    procedure :: toGPST  ! 转换到 GPS 时间
    procedure :: toBDT   ! 转换到北斗时间
endtype
```

支持的时间系统

标识符	常量值	时间系统	说明
`TIME_TYPE_UTC`	1	UTC	协调世界时（默认类型）
`TIME_TYPE_TT`	2	TT	地球力学时（原 TDT）
`TIME_TYPE_UT1`	3	UT1	世界时 1（含极移修正）
`TIME_TYPE_BDT`	4	BDT	北斗时
`TIME_TYPE_GPST`	5	GPST	GPS 时间

MJD 简介

约化儒略日（MJD）与儒略日（JD）的关系为：

$$\mathrm{MJD} = \mathrm{JD} - 2400000.5$$

MJD 的 0 点对应格林尼治子夜（而非正午），在卫星轨道计算中更为便利。例如，J2000.0 的历元为：

$$\mathrm{JD} = 2451545.0, \quad \mathrm{MJD} = 51544.5$$

3.3 时间初始化

`init` 方法支持三种输入方式，根据参数个数自动判定：

方式一：MJD 直接输入

```
type(type_t) :: t
call t%init(51544.5d0)          ! J2000.0
```

传入一个双精度浮点数，解释为完整 MJD 值。

方式二：日-秒分别输入

```
type(type_t) :: t
call t%init(51544d0, 43200d0)   ! MJD 日数 + 当日秒数（正午）
```

传入两个双精度数：第一个为 MJD 整数日数，第二个为当日秒数（0 ≤ sec < 86400）。

方式三：日历日期输入

```
type(type_t) :: t
call t%init(2000d0, 1d0, 1d0, 12d0, 0d0, 0d0)  ! 2000-01-01 12:00:00
```

传入六个双精度浮点数：年、月、日、时、分、秒。内部通过 Gregorian→JD→MJD 路径转换为 MJD 表示。

3.4 时间系统转换

所有转换均为 type-bound procedure，会自动修改对象内部的 `ttype` 和数值。

UTC ↔ TT

地球力学时（TT）与协调世界时（UTC）的差别在于闰秒和相对论修正：

$$\mathrm{TAI} = \mathrm{UTC} + \Delta\mathrm{LS}, \quad \mathrm{TT} = \mathrm{TAI} + 32.184\,\mathrm{s}$$

其中 ΔLS 为闰秒数（截至 2017 年为 37 秒）。

```
type(type_t) :: t
call t%init(2025d0, 1d0, 1d0, 0d0, 0d0, 0d0)  ! UTC
call t%toTT()    ! 转换为 TT
call t%toUTC()   ! 转回 UTC
```

GPS 时间与北斗时间

GPS 时间和北斗时间的内部表示为周数（week）和周内秒（sow, seconds of week）：

```
type(type_t) :: t
real(8) :: week, sow

call t%init(2025d0, 1d0, 1d0, 0d0, 0d0, 0d0)   ! UTC
call t%toGPST(week, sow)    ! 转为 GPS 时间，获取周数+周内秒
call t%toBDT(week, sow)     ! 转为北斗时间
```

北斗时（BDT）与 GPS 时间的固定偏差：BDT = GPST − 14 秒。北斗时起始历元为 2006-01-01 00:00:00 UTC（MJD 53736），对应 BDT 第 0 周。

转换路线

时间转换的实现采用中间路过机制。例如，GPST → TT 的转换实际上经过 BDT 中转：

- UTC → TT：直接利用闰秒表
- BDT → TT：BDT 历元偏移 (MJD 53736) + 65.184 秒
- GPST → TT：先转 BDT（减 9492 天 + 14 秒），再转 TT
- UT1 → UTC：当前为预留接口（显示 "unfinished part"）

> **注意**：UT1 相关的转换（`toUT1`、UT1→UTC、UT1→BDT 等）当前尚未完全实现，调用时会输出警告。

3.5 时间运算

时间差

```
type(type_t) :: t1, t2
real(8) :: dt

t1%init(2025d0, 1d0, 2d0, 0d0, 0d0, 0d0)
t2%init(2025d0, 1d0, 1d0, 0d0, 0d0, 0d0)
dt = timeMinus(t1, t2)    ! 返回时间差，单位：天
! dt = 1.0
```

`timeMinus` 返回以天为单位的时间差。`timeAdd` 则用于添加时间间隔：

```
type(type_t) :: t, t2
t%init(2025d0, 1d0, 1d0, 0d0, 0d0, 0d0)
t2 = timeAdd(t, 1.0d0)          ! 加 1 天
t2 = timeAdd(t, 0d0, 3600d0)    ! 加 0 天 + 3600 秒
```

两种重载形式：

- `timeAdd(t, dt)` — `dt` 以天为单位（可正可负）
- `timeAdd(t, days, secs)` — 分别指定天数增量与秒数增量

3.6 日期格式化

```
type(type_t) :: t
character(len=28) :: str

call t%init(2025d0, 3d0, 15d0, 14d0, 30d0, 0d0)
str = t%toDate()                ! "2025-03-15 14:30:00.000"
```

`toDate()` 返回格式为 `YYYY-MM-DD HH:MM:SS.sss` 的字符串，分辨率为毫秒。仅在 `ttype == TIME_TYPE_UTC` 时可用。

3.7 底层转换函数

除了 type-bound procedure 外，`module_time` 还提供了一些直接使用的函数：

函数	输入	输出	说明
`date2MJD(y,m,d,sec, day,s)`	日历日期	MJD (日数, 秒)	日历→MJD 转换
`UTC2TT(utc, tt)`	UTC (MJD)	TT (MJD)	UTC→TT
`TT2UTC(tt, utc)`	TT (MJD)	UTC (MJD)	TT→UTC
`UTC2TT_daySec(d1,s1, d2,s2)`	UTC (日,秒)	TT (日,秒)	日-秒形式的 UTC→TT
`TT2UTC_daySec(d1,s1, d2,s2)`	TT (日,秒)	UTC (日,秒)	日-秒形式的 TT→UTC
`TT2BDT_daySec(d,s, wk,sow)`	TT (日,秒)	BDT (周,周内秒)	TT→北斗时
`BDT2TT_daySec(wk,sow, d,s)`	BDT (周,周内秒)	TT (日,秒)	北斗时→TT
`BDT2TT(wk,sow, TT)`	BDT (周,秒)	TT (MJD)	北斗时→TT
`UTC2UT1(utc, dut1, ut1)`	UTC (MJD), DUT1	UT1 (MJD)	UTC→UT1
`JD2date(DJ1, DJ2, Y,M,D,FD, J)`	JD 两部分	年/月/日/日小数	IAU SOFA 原生 JD→日期

闰秒处理

`UTC2TT_daySec` 内置了 IERS 闰秒表：

UTC 起始 MJD	闰秒数 ΔLS
57754	37 (2017-01-01 至今)

UTC 起始 MJD	闰秒数 ΔLS
57204	36
56109	35
54832	34
53736	33
51179	32
50630	31

超出此范围的 UTC 日期将输出错误信息。

3.8 地球定向参数：`module_EOPParam`

`module_EOPParam` 管理与地球自转相关的三个关键参数：

参数	说明	用途
`Xp`，`Yp`	极移分量	ECI→ECF 变换的极移矩阵
`DUT1`	UT1 − UTC 差值	UTC→UT1 转换

主要操作

```
call syncEOP(t)     ! 同步 EOP 参数到给定历元 t (MJD)
```

`syncEOP` 在每次坐标变换前自动调用，确保使用的 EOP 值与当前历元匹配。EOP 数据来自 `data/data_op/` 目录下的 EOP 表文件。

在 `module_space` 的 `HG_get` 和 `HG_get_sofa_06a` 中，可以清楚看到 EOP 的使用流程：

```
call syncEOP(t)                    ! 1. 同步 EOP
call UTC2UT1(UTC, DUT1, UT1)       ! 2. UTC → UT1
call UTC2TT(UTC, TT)               ! 3. UTC → TT
! 4. 调用 IAU 模型计算旋转矩阵 HG
```

3.9 使用建议

1. **默认使用 UTC**：初始化时间对象时，日历输入的默认时间类型是 UTC；MJD 输入需自行确保类型正确
2. **先转换再做运算**：时间差计算 `timeMinus` 直接对 `day` 和 `sec` 字段操作，不区分时间类型——建议先统一到同一时间系统再比较
3. **闰秒局限**：内置闰秒表截止到 2017 年的第 37 次闰秒。如需处理更晚的日期，必须在 `UTC2TT_daySec` 中更新闰秒数据
4. **UT1 不完整**：`toUT1()` 和相关转换当前为未实现状态，如需使用 UT1→UTC，应从 EOP 数据中手动获取 DUT1
5. **SOFA 辅助**：`JD2date` 是一个完整的 IAU SOFA 实现，可用于高精度 JD→日历日期转换，精度优于自行实现的版本
   \newpage

第4章 坐标系统

4.1 概述

坐标系的定义与变换是卫星轨道力学的基础环节。Atlas 通过 `module_space` 和 `module_cartesian` 提供了一套完整的坐标系管理与变换框架，支持天球参考系（ICRS/J2000）、地固参考系（ITRS/ECF）、地平坐标系等多种参考框架之间的变换。

对应模块：`module_space`（Level 3），`module_cartesian`（Level 3）

4.2 坐标框架类型

Atlas 定义了以下坐标框架（`module_cartesian`）：

常量	值	框架	说明
`COORD_ICRS`	1	ICRS / J2000	国际天球参考系（地心惯性系）
`COORD_ITRS`	2	ITRS / ECF	国际地球参考系（地心地固系）
`COORD_HORIZON`	3	HORIZON	当地地平坐标系（站心系）

在 `module_space` 中，术语使用 ECI（Earth-Centered Inertial）和 ECF（Earth-Centered Fixed），实质上分别对应 ICRS 和 ITRS。

4.3 笛卡尔坐标对象：`type_cartesian`

`module_cartesian` 将位置、速度、参考框架和时间封装为一个面向对象的数据结构：

```
type type_cartesian
    real(8) :: position(3)    ! 位置矢量 (km), SI 单位
    real(8) :: velocity(3)    ! 速度矢量 (km/s), SI 单位
    integer :: reference      ! 参考框架标识
    type(type_t) :: t         ! 关联时刻
contains
    procedure :: init         ! 初始化
    procedure :: ICRS2ITRS    ! ICRS → ITRS
    procedure :: ITRS2ICRS    ! ITRS → ICRS
    procedure :: ITRS2Horizon ! ITRS → 地平坐标系
    procedure :: Horizon2ITRS ! 地平坐标系 → ITRS
    procedure :: getReferenceName  ! 获取框架名称字符串
endtype
```

> **注意**：`type_cartesian` 使用 SI 单位（km, km/s），而库内部计算使用归一化单位。在 `type_satellite` 等内部对象中需注意单位转换。

初始化

```
type(type_cartesian) :: cart
type(type_t) :: t
real(8) :: r(3), v(3)

call t%init(2025d0, 1d0, 1d0, 0d0, 0d0, 0d0)
r = (/7000d0, 0d0, 0d0/)       ! 位置 (km)
v = (/0d0, 7.5d0, 0d0/)        ! 速度 (km/s)

call cart%init(r, v, COORD_ICRS, t)
```

4.4 ECI ↔ ECF 变换

ECI（天球惯性系）与 ECF（地固系）之间的坐标变换是卫星轨道计算中最常用的操作。Atlas 支持两种 IAU 模型：

IAU1980 模型

基于经典的岁差-章动-自转分解，通过 `HG_get` 计算旋转矩阵：

```
HG = EP · ER · NR · PR
```

其中：

- **PR**（岁差矩阵）：基于 IAU1976 岁差理论
- **NR**（章动矩阵）：20 项主章动项
- **ER**（地球自转矩阵）：格林尼治恒星时 + Δμ 章动修正
- **EP**（极移矩阵）：基于 EOP 极移参数

IAU2006A 模型

通过内置 SOFA 的 `iau_C2T06A` 实现 CIO-based（天球中间原点）变换，精度更高。

变换子程序

`module_space` 提供了多组变换子程序，覆盖不同 IAU 模型和是否含速度变换：

子程序	功能	速度变换
`ECI2ECF_IAU1980_position(utc, rECI, rECF)`	ECI→ECF	仅位置
`ECI2ECF_IAU1980_positionVelocity(utc, rECI, vECI, rECF, vECF)`	ECI→ECF	位置+速度
`ECF2ECI_IAU1980_position(utc, rECF, rECI)`	ECF→ECI	仅位置
`ECF2ECI_IAU1980_positionVelocity(utc, rECF, vECF, rECI, vECI)`	ECF→ECI	位置+速度
`ECF2ECI_IAU2006A_position(utc, rECF, rECI)`	ECF→ECI (IAU2006A)	仅位置

其中 `utc` 为 MJD (UTC)，位置和速度使用归一化单位。

使用示例

```
! 直接使用 module_space 的子程序（归一化单位）
real(8) :: utc, rECI(3), vECI(3), rECF(3), vECF(3)

utc = 51544.5d0  ! J2000.0
rECI = (/1.1d0, 0d0, 0d0/)   ! 归一化单位 (Ae)
vECI = (/0d0, 0.5d0, 0d0/)   ! 归一化单位 (Ae/TU)

call ECI2ECF_IAU1980_positionVelocity(utc, rECI, vECI, rECF, vECF)

! 使用 type_cartesian 对象（SI 单位）
type(type_cartesian) :: cart
! ... 初始化 cart 为 ICRS ...
call cart%ICRS2ITRS()   ! 自动使用内部时间
```

变换流程

每次变换的内部流程为：

1. **同步 EOP**：`call syncEOP(utc)` — 获取极移 `Xp, Yp` 和 `DUT1`

2. **计算 UT1 和 TT**：`UTC2UT1` 和 `UTC2TT`
3. **构建旋转矩阵 HG**：通过 IAU1980 岁差-章动-自转链或 IAU2006 CIO 方法
4. **应用旋转**：`rECF = HG · rECI`

速度变换时，还需计算 HG 的时间导数：

```
vECF = HG · vECI + Ḣ G · rECI
```

其中 `Ḣ G` 通过有限差分近似（时间步长 0.1 秒）。

4.5 地平坐标系

当地地平坐标系（Horizon Frame）以地面站为原点：

- **x 轴**：指向北方（水平面内）
- **y 轴**：指向天顶
- **z 轴**：指向东方（水平面内）

变换矩阵

从 ECF 到地平坐标系的旋转矩阵 ZR 定义为：

```
ZR = R_y(π) · R_z(3π/2) · R_x(-(π/2 - ϕ)) · R_z(λ - π/2)
```

其中 λ 为测站经度，φ 为测站纬度。

```
real(8) :: longi, lati, rECF(3), rHorizon(3)

! ECF → 地平
call ECF2horizon_position(longi, lati, rECF, rHorizon)

! 地平 → ECF
call horizon2ECF_position(longi, lati, rHorizon, rECF)
```

使用 type_cartesian

```
type(type_cartesian) :: cart
! ... 初始化 cart 为 ITRS ...
call cart%ITRS2Horizon(longitude, latitude)    ! ITRS → 地平
call cart%Horizon2ITRS(longitude, latitude)    ! 地平 → ITRS
```

> **注意**：地平坐标系的速度变换目前未实现（变换后速度置零）。

4.6 地面站坐标计算

大地坐标 → ECF 直角坐标

```
subroutine station_lla2xyz(longitude, latitude, altitude, position)
! 输入：
!   longitude    大地经度 (rad)，东经为正
!   latitude     大地纬度 (rad)，北纬为正
!   altitude     大地高 (Ae，归一化单位)
! 输出：
!   position(3) ECF 直角坐标 (Ae)
```

计算使用 WGS-84 参考椭球（扁率 f = 1/298.257223563），公式为：

$$
\begin{aligned}
N &= \left[\cos^2\varphi + (1-f)^2\sin^2\varphi\right]^{-1/2} \\
x &= (N+h)\cos\varphi\cos\lambda \\
y &= (N+h)\cos\varphi\sin\lambda \\
z &= \left[N(1-f)^2 + h\right]\sin\varphi
\end{aligned}
$$

`module_space` 中另有一个等价的 `get_station_position` 子程序，输出归一化单位。

4.7 NTH 坐标系

NTH（Normal-Tangential-Horizontal，法向-切向-水平）是描述轨道局部运动的常用坐标系：

- **N（法向）**：沿 `(v × h)` 方向，指向轨道面法向

- **T（切向）**：沿速度矢量方向
- **H（水平）**：沿轨道角动量 `h = r × v` 方向

```
real(8) :: r(3), v(3), M_NTH2ECI(3,3), M_ECI2NTH(3,3)

call NTH2ECI(r, v, M_NTH2ECI)    ! NTH → ECI 变换矩阵
call ECI2NTH(r, v, M_ECI2NTH)    ! ECI → NTH 变换矩阵
! M_ECI2NTH = transpose(M_NTH2ECI)
```

这在推力建模（`module_thruster`）和轨道机动分析中非常有用。

4.8 月心坐标系

`module_space` 还提供了地月会合系和月心固定/惯性系之间的变换：

子程序	功能
`ECI2EarthMoonsynodic(r,v,rm,vm, rc,vc)`	ECI → 地月会合系
`EarthMoonSynodic2ECI(r,v,rm,vm, rc,vc)`	地月会合系 → ECI
`LCF2LCI(t, rf, ri)`	月心固定 → 月心惯性
`LCI2LCF(t, ri, rf)`	月心惯性 → 月心固定

月心惯性/固定变换使用 JPL 历表第 15 组数据（月球物理天平动），包括 ΩP（月球自转轴进动角）、is（交角）和 Λ（月球自转角）。

4.9 旋转矩阵工具

`module_space` 包含基本的旋转矩阵构造函数和 SOFA 接口：

子程序	功能
`Rxmatrix(theta, a)`	绕 x 轴旋转 θ 角的旋转矩阵
`Rymatrix(theta, a)`	绕 y 轴旋转 θ 角的旋转矩阵

子程序	功能
`Rzmatrix(theta, a)`	绕 z 轴旋转 θ 角的旋转矩阵
`coordinate_reverse(axis, R)`	坐标反向矩阵（绕指定轴 180° 旋转）
`HG_get(t, HG)`	计算 IAU1980 ECI→ECF 旋转矩阵
`HG_get_sofa_06a(t, HG)`	计算 IAU2006A CIO-based 旋转矩阵（SOFA）

4.10 使用建议

1. **框架一致**：使用 `type_cartesian` 的变换方法时，它会检查当前框架是否匹配——不要对已是 ITRS 的对象再次调用 `ICRS2ITRS`
2. **速度变换开销**：含速度的变换需计算 HG 矩阵的时间导数（通过两次 HG 计算做有限差分），比纯位置变换耗时约两倍。当速度为零时，`type_cartesian` 的变换方法会自动跳过速度计算
3. **EOP 数据**：确保 `data/data_op/` 下有最新的章动系数文件（`Nutation.dat`）和 EOP 表，否则 IAU1980 变换可能失败
4. **SOFA 已内置**：IAU2006A 变换调用 SOFA 的 `iau_C2T06A`。SOFA 源码位于 `sofa/`，随库一起编译，无需任何外部依赖；对精度要求不高的场合也可改用 IAU1980 版本
5. **单位注意**：`module_space` 中的子程序使用归一化单位，`type_cartesian` 使用 SI 单位——混合使用时注意在边界处转换
   \newpage

第5章 轨道根数

5.1 概述

轨道根数（Orbital Elements）是描述卫星在轨道上运动状态的六维参数。Atlas 支持四类轨道根数，并提供它们之间的两两转换。正确选择和使用轨道根数类型对数值稳定性和计算精度有重要影响。

对应模块：`module_elements`（Level 0）

5.2 四类轨道根数

编码	常量	名称	分量	
1	`STATE_TYPE_KEPLERIAN`	Keplerian 开普勒根数	`(a, e, i, Ω, ω, M)`	轨道
2	`STATE_TYPE_CARTESIAN`	Cartesian 笛卡尔根数	`(x, y, z, vx, vy, vz)`	直接
3	`STATE_TYPE_NSG1`	第一类非奇异根数	`(a, i, Ω, ξ, η, λ)`	小倾
4	`STATE_TYPE_NSG2`	第二类非奇异根数（春分点根数）	`(a, h, k, p, q, λ)`	小倾

5.2.1 Keplerian 开普勒根数

分量	符号	说明	单位（SI）
1	`a`	半长轴	km
2	`e`	偏心率	—
3	`i`	轨道倾角	度
4	`Ω`	升交点赤经 (RAAN)	度
5	`ω`	近地点幅角	度
6	`M`	平近点角	度

六个分量的 SI 单位和归一化单位关系：

分量	SI 单位	归一化单位
`a`	km	÷ `length_unit`
`e`	无量纲	不变
`i, Ω, ω, M`	度	× `deg2rad`

奇异性问题：当 `e = 0`（圆轨道）时，近地点无定义 → `ω` 和 `M` 不唯一；当 `i = 0` 或 `i = π`（赤道轨道）时，升交点无定义 → `Ω` 和 `ω` 不唯一。在精密定轨和长期预报中，这些奇异性会导致协方差矩阵病态。

5.2.2 Cartesian 笛卡尔根数

分量	符号	说明	单位（SI）
1–3	`r = (x, y, z)`	位置矢量	km
4–6	`v = (vx, vy, vz)`	速度矢量	km/s

Cartesian 根数是数值积分的最直接形式，没有奇异性问题。但它缺少直观的轨道几何含义，且在使用普通数值积分器时可能导致能量/角动量漂移。

5.2.3 NSG1：第一类非奇异根数

分量	表达式	说明
1	`a`	半长轴
2	`i`	轨道倾角
3	`Ω`	升交点赤经
4	`ξ = e cos ω`	偏心率在近地点方向的分量
5	`η = e sin ω`	偏心率在正交方向的分量
6	`λ = M + ω`	平经度

特点：通过将 `e` 和 `ω` 组合为 `ξ` 和 `η`，消除了小偏心率（`e → 0`）时的奇异性。但仍保留 `i` 和 `Ω`，不适用于赤道轨道。

5.2.4 NSG2：第二类非奇异根数（春分点根数）

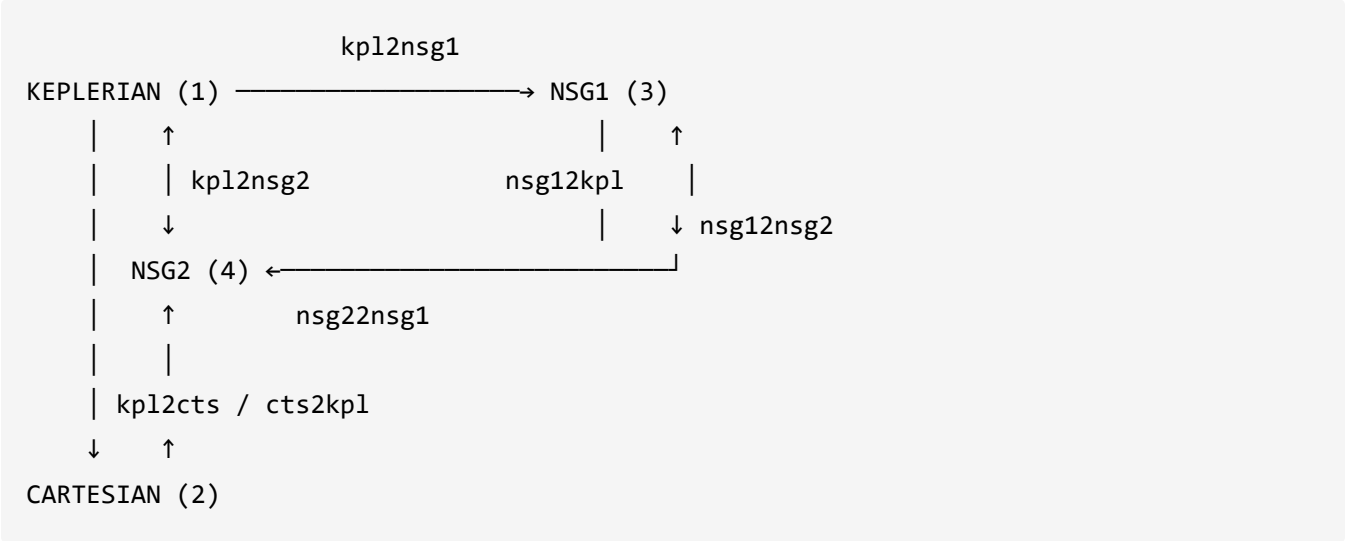
分量	表达式	说明
1	`a`	半长轴
2	`h = e cos(Ω + ω)`	偏心率在春分点方向的分量
3	`k = e sin(Ω + ω)`	偏心率在正交方向的分量
4	`p = sin(i/2) cos Ω`	倾角在 x 方向的分量

分量	表达式	说明
5	`q = sin(i/2) sin Ω`	倾角在 y 方向的分量
6	`λ = Ω + ω + M`	平经度

特点：同时消除了小偏心率和小倾角的奇异性，是精密定轨（POD）和状态转移矩阵（STM）计算中最推荐的根数类型。

5.3 根数转换网络

`module_elements` 实现了四类根数之间的转换。转换关系如下：



所有转换可通过 Keplerian 根数中转：

转换方向	路径
NSG1 → Cartesian	NSG1 → KPL → CTS（`nsg12cts`）
NSG2 → Cartesian	NSG2 → KPL → CTS（`nsg22cts`）
Cartesian → NSG1	CTS → KPL → NSG1（`cts2nsg1`）
Cartesian → NSG2	CTS → KPL → NSG2（先 `cts2kpl` 再 `kpl2nsg2`）
NSG1 ↔ NSG2	通过 KPL 中转（`nsg12nsg2` / `nsg22nsg1`）

Keplerian → Cartesian

`kpl2cts` 通过求解开普勒方程得到偏近点角 E，进而计算位置和速度：

```
! kpl(6) = (/a, e, i, Ω, ω, M/)
call kpl2cts(kpl, cts)
! cts(1:3) = r, cts(4:6) = v（归一化单位）
```

算法：牛顿迭代求解 `E - e sin(E) = M`（5 次迭代），然后用 P、Q 向量构造位置速度。

Cartesian → Keplerian

`cts2kpl` 从位置速度矢量反算开普勒根数：

```
call cts2kpl(rv, elements)  ! rv = (/r, v/), elements = (/a,e,i,Ω,ω,M/)
```

该方法在归一化单位下工作（引力常数 μ = 1），计算步骤为：

1. 由能量积分求 `a`
2. 由偏心率矢量求 `e`
3. 由角动量方向求 `i, Ω`
4. 求解开普勒方程得到 `E, M`

NSG1 与 NSG2 的特殊函数

`module_elements` 为非奇异根数提供了特殊的函数：

```
! 从 NSG1 根数解算位置
call nsg12rut(nsg1, r, u, ut)
! r = a(1 - ξ cos(ũ) - η sin(ũ))
! u = ω + f (真纬度角), ũ = ω + E (偏纬度角)

! 从 NSG2 根数解算位置
call nsg22rut(nsg2, r, u, ut)
! 同样的公式形式，但根数定义不同
```

广义开普勒方程 `λ = ũ - ξ sin(ũ) + η cos(ũ)` 通过定点迭代求解（最多 50 次，容差 10^{-12}）。

5.4 单位转换

`module_elements` 还提供了 SI ↔ 归一化单位的转换：

```
! Keplerian: SI → Normal
call kpl_SI2normal(x0, x1)
! x1(1) = x0(1)/length_unit        (a)
! x1(2) = x0(2)                     (e: 不变)
! x1(3:6) = x0(3:6)*deg2rad        (角度 → 弧度)

! Keplerian: Normal → SI
call kpl_normal2SI(x1, x0)

! Cartesian: SI → Normal
call cts_SI2normal(x0, x1)
! x1(1:3) = x0(1:3)/length_unit
! x1(4:6) = x0(4:6)/length_unit*time_unit

! Cartesian: Normal → SI
call cts_normal2SI(x1, x0)
```

5.5 二体问题解析传播

对于无摄二体问题（Kepler 运动），`module_elements` 提供了简化的解析公式：

```
call OP2Body(x0, dt, x1)
! 输入: x0(6) = NSG1 根数（归一化单位）
!       dt   = 时间步长（归一化单位）
! 输出: x1(6) = NSG1 根数
! 公式: x1(6) = x0(6) + a^{-3/2} * dt    （仅更新平经度）
```

这是最简单的二体传播——只有平经度随时间线性增长，其余 5 个根数不变。在归一化单位下，`n = a^{-3/2}`。

5.6 使用建议

1. **默认使用 NSG2**：精密定轨和长期轨道预报推荐使用 NSG2（春分点根数），它避免了偏心率和小倾角的奇异性
2. **Cartesian 用于数值积分**：数值积分器（RK8）以 NSG1/NSG2 为状态量，积完一步后通过根数→Cartesian→力计算→Cartesian→根数的循环实现
3. **Keplerian 用于显示**：开普勒根数最直观，适合在输出和显示时使用
4. **注意单位**：`kpl2cts` 和 `cts2kpl` 在归一化单位下工作（μ = 1），而 `kpl_SI2normal` 等负责 SI 转换
5. **角度范围**：所有角度分量在使用前都应归一化到 `[0, 2π)` 范围
   \newpage

第6章 力模型

6.1 概述

力模型是轨道预报精度的决定因素。Atlas 提供了一套完整的摄动力模型系统，涵盖地球非球形引力、大气阻力、太阳光压、第三体引力、后牛顿效应和推力。

力模型的选择通过 8 维开关数组 `K(1:8)` 和若干物理参数控制，所有设置在对 `NumOP` 的一次调用中传入，实现灵活的力模型组合。

对应模块：`module_Gravity`（Level 4）、`module_atmosphere`（Level 4）、`module_perturbationAspherical`（Level 4）、`module_perturbationSRP`（Level 4）、`module_perturbationThirdBody`（Level 4）、`module_perturbationPN`（Level 4）、`module_thruster`（Level 4）

6.2 力模型开关：K(1:8)

索引	参数	默认值	力模型	说明
K(1)	`k1`	1	田谐项摄动	地球非球形引力高阶部分
K(2)	`k2`	1	月球引力摄动	月球第三体引力

索引	参数	默认值	力模型	说明
K(3)	`k3`	0	月球潮汐形变	地球的月球潮汐响应
K(4)	`k4`	1	太阳引力摄动	太阳第三体引力
K(5)	`k5`	0	太阳潮汐形变	地球的太阳潮汐响应
K(6)	`k6`	0	太阳光压	太阳辐射对卫星的压力
K(7)	`k7`	0	大气阻力	高层大气对卫星的阻力
K(8)	`k8`	0	后牛顿效应	广义相对论 1PN 修正

默认设置（K(1:8) = (1,1,0,1,0,0,0,0)）表示仅考虑地球非球形、日月引力摄动。

6.3 地球非球形引力

6.3.1 引力场对象：`GravityField`

`module_Gravity` 定义了一个通用的引力场对象，支持任意中心天体的球谐展开：

```
type :: GravityField
    character(50) :: name           ! 名称
    character(50) :: centralBody    ! 中心天体
    real(8) :: GM                   ! 引力常数 (m³/s²)
    real(8) :: referenceRadius      ! 参考半径 (m)
    integer :: maxdegree, maxOrder  ! 最大阶次
    real(8), allocatable :: C(:,:)  ! 球谐系数 C_lm
    real(8), allocatable :: S(:,:)  ! 球谐系数 S_lm
contains
    procedure :: init         ! 初始化并从文件加载系数
    procedure :: acceleration ! 计算引力加速度
    procedure :: show         ! 显示引力场信息
endtype
```

6.3.2 球谐展开理论

地球引力势的球谐展开为：

$$U(r,\phi,\lambda)=\frac{GM}{r}\sum_{l=0}^{\infty}\sum_{m=0}^{l}\left(\frac{R_e}{r}\right)^l\bar{P}_{lm}(\sin\phi)[\bar{C}_{lm}\cos m\lambda+\bar{S}_{lm}\sin m\lambda]$$

其中：

- `R_e` 为地球参考半径
- `\bar{P}_{lm}` 为归一化连带勒让德函数
- `\bar{C}_{lm}, \bar{S}_{lm}` 为归一化球谐系数

引力加速度 = `∇U`，分为两个部分：

- **带谐项（Zonal）**：m = 0，仅依赖纬度
- **田谐项（Tesseral）**：m > 0，依赖经纬度

6.3.3 使用方法

```
type(GravityField) :: geoField

! 初始化
call geoField%init("GGM03C", "Earth", 360, 360, "data/data_op/GGM03C.txt")

! 计算加速度
call geoField%acceleration(position, degree, order, acceleration)
```

`acceleration` 方法的输入输出均使用 SI 单位（米、m/s²）。内部计算会转换到归一化球坐标。

6.3.4 田谐项摄动（`module_perturbationAspherical`）

`module_perturbationAspherical` 在预定义的全球重力场模型（默认 GGM03C, 360 阶）基础上计算田谐项摄动加速度。它为每个被积卫星状态计算 `acceleration`，通过 K(1) 开关控制。

6.3.5 月球引力场

Atlas 支持月球中心天体的引力场计算，由 `orderMoon` 参数控制月球引力场阶次（默认 0 = 仅二体模型）。

6.4 大气阻力

`module_atmosphere` 提供两种大气模型：

大气模型类型

模型	`atmosphereModel`	说明
Model 1	1（默认）	椭球分层指数大气模型（周日变化），查表获取密度
Model 2	2	NRLMSISE-00 经验大气模型（需 legacy Fortran 代码）

阻力加速度

```
subroutine atmospherical_drag_perturbation(UTC, rs, rv, Cd, amrDrag, F107, Ap, F)
! 输入:
!   UTC     - MJD (UTC)
!   rs      - 卫星位置，J2000 坐标系 [km]
!   rv      - 卫星速度，J2000 坐标系 [km/s]
!   Cd      - 阻力系数（典型值 2.2）
!   amrDrag- 面积质量比 [m²/kg]（典型值 0.02）
!   F107    - 太阳射电通量 F10.7（典型值 60-240）
!   Ap      - 地磁指数（仅支持 5, 10, 15）
```

阻力加速度公式：

$$\mathbf{F}_{\text{drag}} = -\frac{1}{2} C_D \frac{A}{m} \rho |\mathbf{v}_{\text{rel}}| \mathbf{v}_{\text{rel}}$$

其中：

- `ρ` 为大气密度（由模型给出）
- `v_rel = v_sat - v_atm` 为卫星相对大气的速度
- 大气假定与地球刚体共旋

Model 1：椭球分层指数模型

- 基于 DTM 型椭球大气分层

- 大气密度从预计算表格（`data/data_op/nrlmsise00_cheng/`）插值
- 表格按 Ap（5/10/15）和 F10.7（60-240, 步长 20）分类
- 密度随高度呈指数衰减，含周日变化修正

Model 2：NRLMSISE-00

- 调用 legacy `nrlmsise00_sub.for`
- 当前接口处于预留状态，需完善调用

大气密度计算参数

参数	说明	范围
`Cd`	阻力系数	通常 2.0–2.5
`amrDrag`	A/m 面积质量比	取决于卫星几何
`F107`	太阳 10.7 cm 辐射通量	60–240 sfu
`Ap`	三小时地磁活动指数	5, 10, 15

6.5 太阳光压（SRP）

`module_perturbationSRP` 提供两种光压模型：

模型	`SRP_Model`	说明	参数数
Cannonball	1	球形简化模型	1（`reflectivity`）
ECOM	2	9 参数扩展 CODE 模型	9（`ECOM(1:9)`）

Cannonball 模型

将卫星视为均质球体，光压加速度为：

$$\mathbf{F}_{\mathrm{SRP}} = \kappa \cdot \frac{A}{m} \cdot \rho_s \cdot \left(\frac{1\,\mathrm{AU}}{|\mathbf{r}_{sat} - \mathbf{r}_{sun}|} \right)^2 \cdot \hat{\mathbf{d}}$$

其中：

- `κ = 1 + ε`（反射率修正，ε 为表面反射率）
- `ρ_s = 4.5605 × 10⁻⁶ N/m²`（1 AU 处太阳辐射常数）
- `\hat{d}` 为卫星→太阳方向的单位矢量

ECOM 9 参数模型

扩展 CODE 轨道模型（ECOM）将光压分解为 DYB 三个方向上的傅里叶级数：

```
D-方向（卫星→太阳）：    D₀ + Dc·cos(u) + Ds·sin(u)
Y-方向（太阳×D 方向）：  Y₀ + Yc·cos(u) + Ys·sin(u)
B-方向（D×Y 完成右手系）：B₀ + Bc·cos(u) + Bs·sin(u)
```

其中 `u = ω + f` 为卫星纬度角。9 个参数通过精密定轨解算。

地影模型

Atlas 实现了两种地影判断模型：

`shadowModel`	说明
1	圆柱形地影（默认）：仅判断方向条件
2	锥形地影：更精确的本影/半影判断

地影判断逻辑：

```
cos_ϕ = r̂_sat · r̂_sun
sin_ϕ = √(1 − cos²ϕ)
if (cos_ϕ < 0 .and. sin_ϕ < 1/|r_sat|) → 在地影中（圆柱形）
```

在地影中，光压加速度置为零。

6.6 第三体引力

`module_perturbationThirdBody` 计算太阳、月球和行星对卫星的引力摄动：

$$\mathbf{F}_{3\mathrm{rd}} = -GM_p \left[\frac{\mathbf{r}_{sat} - \mathbf{r}_p}{|\mathbf{r}_{sat} - \mathbf{r}_p|^3} + \frac{\mathbf{r}_p}{|\mathbf{r}_p|^3} \right]$$

所有天体位置通过 `module_planets` 从 JPL DE440 历表获取。

控制参数

参数	默认值	说明
K(2) = `k2`	1	月球引力摄动
K(3) = `k3`	0	月球潮汐形变
K(4) = `k4`	1	太阳引力摄动
K(5) = `k5`	0	太阳潮汐形变
`Mercury` – `Pluto`	0	各行星摄动开关

行星质量比（相对于地球）定义在 `module_const` 中，例如：

天体	质量比
月球	1.2300037×10^{-2}
太阳	332946.04
木星	317.8
金星	0.815

潮汐形变

月球和太阳的潮汐形变通过 K(3) 和 K(5) 控制，使用 `get_F_tide_Liulin` 子程序计算地球对第三体引力场的弹性响应。

6.7 后牛顿效应（Post-Newtonian）

`module_perturbationPN` 实现了广义相对论的一阶后牛顿（1PN）修正。在史瓦西度规下，卫星受到的 1PN 加速度为（由 K(8) 控制）。

这是一个小型模块（~1 KB），实现了相对论摄动的基本框架。

6.8 推力模型

`module_thruster` 提供了轨道机动所需的推力建模：

```
type type_thruster
    ! 比冲、推力范围、方向、燃耗等
endtype
```

推力可以在多个坐标系中定义方向：

- **J2000（惯性系）**：适用于惯性定向推力
- **SBCS（星体系）**：适用于固定于星体的推力器
- **NTH（法-切-水平系）**：适用于轨道机动的自然分解

推力模型通过 `module_orbitPredictionNumerical` 中的 `thisThruster` 变量集成到力模型系统中。

6.9 总加速度合成

在 `NumOP` 中，所有力模型的加速度矢量被矢量叠加：

$$\mathbf{F}_{\text{total}} = \mathbf{F}_{\text{geo}} + \mathbf{F}_{\text{drag}} + \mathbf{F}_{\text{SRP}} + \mathbf{F}_{\text{3rd}} + \mathbf{F}_{\text{PN}} + \mathbf{F}_{\text{thrust}}$$

各力模型按照 K(1:8) 和其他状态开关的配置独立计算后叠加。所有矢量在归一化单位下叠加，最终转换为加速度供数值积分器使用。

6.10 参数总览

参数	类型	默认值	说明
`order`	int	2	地球引力场阶次（GGM03C 最高 360）
`orderMoon`	int	0	月球引力场阶次（0 = 二体）
`Cd`	real	2.2	大气阻力系数

参数	类型	默认值	说明
`amrDrag`	real	0.02	大气阻力面积质量比 (m²/kg)
`F107`	real	130	太阳辐射通量 F10.7 (sfu)
`Ap`	real	15	地磁活动指数
`reflectivity`	real	0.5	光压反射率 ε
`amrSRP`	real	0.02	光压面积质量比 (m²/kg)
`SRP_Model`	int	1	光压模型（1=Cannonball, 2=ECOM）
`ECOM(1:9)`	real	0	ECOM 9 参数
`shadowModel`	int	1	地影模型（1=圆柱, 2=锥形）
\newpage			

第7章 轨道预报

7.1 概述

轨道预报（Orbit Propagation/Orbit Prediction）是 Atlas 的核心功能之一。给定初始轨道状态和力模型配置，数值积分器将卫星状态外推到未来（或过去）的时刻，输出各历元的位置和速度。

Atlas 使用**自适应步长 8(7) 阶 Runge-Kutta-Fehlberg（RKF78）**数值积分器，在归一化单位下工作，支持非奇异根数（NSG1）和笛卡尔坐标（Cartesian）两种状态表示。

对应模块：`module_orbitPredictionNumerical`（Level 4）

7.2 主入口：`NumOP`

```
subroutine NumOP(K_, order_, &
    Cd_, amrDrag_, F107_, Ap_, &
    reflectivity_, amrSRP_, &
    SRP_model_, ECOM_, &
    step_, &
    orderMoon_, Mercury_, Venus_, Mars_, Jupiter_, Saturn_, Uranus_, Pluto_, &
    eleType_, satNum_, &
    x0, t0, dt, x1)
```

输入参数

参数	类型	说明
`K_(8)`	int	力模型开关（见第6章）
`order_`	int	地球引力场阶次
`Cd_`	real	大气阻力系数
`amrDrag_`	real	阻力面积质量比 (m²/kg)
`F107_`	real	太阳辐射通量 F10.7
`Ap_`	real	地磁活动指数
`reflectivity_`	real	光压反射率
`amrSRP_`	real	光压面积质量比 (m²/kg)
`SRP_model_`	int	光压模型选择 (1=Cannonball, 2=ECOM 9参数)
`ECOM_(9)`	real	ECOM 9 参数
`step_`	real	固定步长（天），= 0 时使用自适应步长
`orderMoon_`	int	月球引力场阶次
`Mercury_` – `Pluto_`	int	各行星引力开关 (0/1)

参数	类型	说明
`eleType_`	int	输入状态类型 (1=KPL, 2=CTS, 3=NSG1, 4=NSG2)
`satNum_`	int	卫星编号
`x0(6)`	real	初始状态（归一化单位）
`t0`	real	初始时刻（MJD，归一化单位）
`dt`	real	预报时间跨度（天，SI 单位）
输出		
`x1(6)`	real	终了状态（归一化单位，与 x0 同类型）

输入/输出状态类型

状态在内部统一转换为 NSG1 根数进行积分（如输入为 Keplerian 或 Cartesian，会先转换为 NSG1）。积分完成后，按输入类型转回输出：

`eleType_`	输入→内部	积分	内部→输出
1 (KPL)	KPL → NSG1	NSG1	NSG1 → KPL
2 (CTS)	直接用 PV 积分	PV（Cartesian）	直接输出 PV
3 (NSG1)	直接积分	NSG1	直接输出 NSG1
4 (NSG2)	预留	—	—

安全保护

输入状态在积分前会检查轨道近地点高度。若 `e < 1` 且近地点低于 100 km，预报终止（状态置零并输出警告）。

7.3 数值积分器：RKF78

Atlas 采用 **Runge-Kutta-Fehlberg 7/8 阶嵌入方法**（RKF78）。这是一种 13 级显式 Runge-Kutta 方法，同时提供 8 阶和 7 阶近似解，二者之差用于自适应步长控制。

算法结构

```
k₀ = h·f(t₀, y₀)
k₁ = h·f(t₀ + α₁h, y₀ + β₁₀k₀)
k₂ = h·f(t₀ + α₂h, y₀ + β₂₀k₀ + β₂₁k₁)
...
k₁₂ = h·f(t₀ + α₁₂h, y₀ + β₁₂₀k₀ + ... + β₁₂₁₁k₁₁)

y_8th = y₀ + Σ c_i·k_i     (8阶解)
y_7th = y₀ + Σ ĉ_i·k_i     (7阶解)
deviation = ||y_8th - y_7th||
```

系数 α_i，β_{ij}，c_i，$\hat{c}_i$ 为 Fehlberg 经典系数表，硬编码在 `rkf78_forOP` 子程序中。

自适应步长控制

```
if (deviation <= error_max .or. fixedStep) then
    t = t + step
    if (error_max >= deviation*100 .and. .not. fixedStep) then
        step = step * 2.0d0           ! 精度充足→扩大步长
    endif
    x0 = x1
else
    step = step / 2.0d0               ! 精度不足→缩小步长
endif
```

- 容许误差 `error_max = 1.5 × 10⁻¹³`（归一化单位）
- 当偏差小于容许误差的 1/100 时，步长翻倍
- 当偏差超限时，步长减半并重新计算当前步
- 固定步长模式（`stepLength ≠ 0`）跳过自适应逻辑

右端函数计算

积分器的右端函数 `f(t, y)` 在 `module_orbitPredictionNumerical` 内部定义，计算流程为：

1. **NSG1 → Cartesian**：将 NSG1 状态转为位置速度
2. **调用各力模型**：根据 K(1:8) 开关依次计算各摄动加速度

3. **加速度叠加**：`a_total = Σ F_i`
4. **Cartesian→NSG1**：将总加速度转回 NSG1 状态的变化率

每个积分步内可能多次调用右端函数（RKF78 有 13 个级），因此力模型的高效实现对整体性能至关重要。

7.4 `generateTrajectory` 接口

`type_satellite` 提供了面向用户的轨迹生成接口：

```
type(type_satellite) :: sat
type(type_t), allocatable :: epochs(:)

! 生成在指定历元数组上的轨迹
call sat%generateTrajectory(epochs)
```

内部实现对 `epochs` 中的每个时刻依次调用 `NumOP`，将完整的轨迹保存在卫星对象内部。然后可以通过 `savetrajectory` 输出到文件。

7.5 两种积分模式

非奇异根数模式（`usePV=0`）

- 状态量：6 维 NSG1 根数 `(a, i, Ω, ξ, η, λ)`
- 优点：长时间弧段下数值稳定性好，能量和角动量守恒性好
- 适用范围：偏心率不太大的椭圆轨道

笛卡尔模式（`usePV=1`）

- 状态量：6 维位置速度 `(x, y, z, vx, vy, vz)`
- 优点：不经过根数转换，适用于任意轨道类型
- 缺点：积分步数增多，能量漂移累积较大

两种模式在 `prediction` 子程序中通过 `usePV` 参数切换。

7.6 使用示例

```
! 基本轨道预报
real(8) :: K(8), x0(6), x1(6), t0, dt

! 设置力模型：仅日月引力
K = (/1, 1, 0, 1, 0, 0, 0, 0/)

! 初始状态：NSG1 根数（归一化单位）
x0 = (/1.1d0, 0.5d0, 1.0d0, 0.01d0, 0.0d0, 0.0d0/)
t0 = 51544.5d0            ! J2000.0 MJD
dt = 1.0d0                ! 预报 1 天

call NumOP(K, 2, &
    2.2d0, 0.02d0, 130d0, 15d0, &   ! 大气参数
    0.5d0, 0.02d0, &                 ! 光压参数
    1, (/0d0,0,0,0,0,0,0,0,0/), &   ! SRP model + ECOM
    0d0, &                           ! 自适应步长
    0, 0,0,0,0,0,0,0, &             ! 行星引力
    3, 1, &                          ! NSG1, 卫星编号
    x0, t0, dt, x1)

! x1 为预报后的 NSG1 根数
```

7.7 性能注意事项

1. **引力场阶次**：`order` 参数对性能影响最大——360 阶球谐展开每步约需计算 65,000 个勒让德函数值
2. **自适应步长**：低轨卫星（~400 km）典型步长约为 10-60 秒；高轨卫星（GPS, ~20000 km）步长可达 300-600 秒
3. **第三体引力**：日月引力计算相对廉价（需调用 JPL 历表插值）；多行星同时开启会显著增加 JPL 历表查询开销
4. **潮汐形变**：月球和太阳的潮汐形变计算（K(3), K(5)）开销较大，非精密应用建议关闭
5. **内存**：`module_orbitPredictionNumerical` 使用 `private` 模块变量存储配置，集成到 `type_satellite` 的轨迹生成时注意状态隔离
   \newpage

第8章 卫星对象

8.1 概述

`type_satellite` 是 Atlas 库的中心领域对象，封装了卫星轨道计算所需的所有信息：初始状态、力模型配置、传播方法、轨迹数据等。配合 `type_satelliteScenario` 多星场景容器，可以实现复杂的多星多站仿真任务。

对应模块：`module_satellite`（Level 5），`module_satelliteScenario`（Level 5）

8.2 `type_satellite` 结构

```
type type_satellite
    integer :: number              ! 卫星编号
    character(100) :: name         ! 卫星名称
    real(8) :: mass                ! 质量（kg）
    type(type_trajectory) :: states ! 初始状态（时刻 + 6 维轨道根数）
    type(type_opMod) :: opMod       ! 传播设置（力模型参数）
    integer :: eleType             ! 轨道根数类型（1=KPL, 2=CTS, 3=NSG1, 4=NSG2）
    type(type_trajectory), allocatable :: trajectory(:) ! 生成的历史轨迹
    integer :: nEph                ! 轨迹点数
    character(10) :: propagator    ! 传播方法: 'numerical', 'two-body', 'SGP4'
    type(type_thruster) :: thruster ! 推力器模型
contains
    procedure :: init              ! 初始化
    procedure :: addThruster       ! 添加推力器
    procedure :: generateTrajectory ! 生成轨迹
    procedure :: savetrajectory    ! 保存轨迹到文件
    procedure :: readtrajectory    ! 从文件读取轨迹
    procedure :: setTrajectory     ! 从矩阵设置轨迹
    procedure :: getStates         ! 插值获取某时刻轨道根数
    procedure :: getPV             ! 插值获取某时刻笛卡尔位置速度
    procedure :: propagate         ! 传播卫星状态到新历元
    procedure :: show              ! 显示卫星信息
endtype
```

8.3 初始化

基本初始化

```
type(type_satellite) :: sat
type(type_t) :: t0
real(8) :: x0(6)

call t0%init(2025d0, 1d0, 1d0, 0d0, 0d0, 0d0)  ! 2025-01-01 00:00:00 UTC

! Keplerian 根数输入 (a, e, i, Ω, ω, M) — SI 单位
x0 = (/7000.0d0, 0.01d0, 30.0d0, 120.0d0, 45.0d0, 0.0d0/)

call sat%init(t0, x0, &
    number = 1, &
    name = "TestSat", &
    mass = 500.0d0, &
    eleType = 1, &            ! Keplerian
    propagator = "numerical", &
    order = 8)                ! 引力场阶次
```

力模型配置：`type_opMod`

`type_opMod` 是 `type_satellite` 的内嵌类型，定义传播所需的全部力模型参数：

```
type :: type_opMod
    ! 大气阻力
    real(8) :: Cd = 2.2d0                ! 阻力系数
    real(8) :: amrDrag = 0.02d0          ! 阻力面积质量比 (m²/kg)
    real(8) :: F107 = 130d0              ! 太阳辐射通量
    real(8) :: Ap = 15d0                 ! 地磁活动指数
    ! 太阳光压
    integer :: SRP_Model = 1             ! 1=Cannonball, 2=ECOM
    real(8) :: reflectivity = 0.0d0      ! 反射率 (0-1)
    real(8) :: amrSRP = 0.02d0           ! 光压面积质量比 (m²/kg)
    real(8) :: ECOM(9) = 0d0             ! ECOM 9 参数
    ! 力模型
    integer :: order = 2                 ! 地球引力场阶次
    integer :: K(8) = 0                  ! 力模型开关
    integer :: orderMoon = 0             ! 月球引力场阶次
    integer :: Mercury ... Pluto = 0     ! 行星引力开关
    ! 积分器
    integer :: usePV = 0                 ! 0=NSG1, 1=Cartesian
    logical :: showProgressBar = .false. ! 显示进度条
    real(8) :: step = 0d0                ! 固定步长（0=自适应）
endtype
```

设置力模型

```
sat%opMod%K = (/1, 1, 0, 1, 0, 0, 0, 0/)    ! 日月+非球形
sat%opMod%order = 8                          ! 8×8 引力场
sat%opMod%Cd = 2.2d0
sat%opMod%amrDrag = 0.02d0
sat%opMod%usePV = 0                          ! 使用 NSG1
```

三种传播方法

`propagator`	说明	适用场景
`'numerical'`	完整数值积分 (RKF78 + 全摄动模型)	高精度预报、精密定轨
`'two-body'`	解析二体运动 (仅平经度变化)	快速预览、粗精度分析

propagator	说明	适用场景
'SGP4'	简化普适摄动模型 (预留)	TLE 兼容

8.4 轨迹生成

generateTrajectory

```
type(type_t), allocatable :: epochs(:)
integer :: i

! 生成每小时一个历元，共 24 小时
allocate(epochs(25))
do i = 1, 25
    epochs(i) = timeAdd(t0, dble(i-1)/24d0)
enddo

call sat%generateTrajectory(epochs)
```

内部调用 NumOP 对每个历元进行传播，结果存储于 sat%trajectory(:) 中。

savetrajectory

将历史轨迹保存到文件：

```
! 保存为 Cartesian 格式（SI 单位）
call sat%savetrajectory(STATE_TYPE_CARTESIAN, "output.txt", freeEph=.true.)

! 保存为 Keplerian 格式
call sat%savetrajectory(STATE_TYPE_KEPLERIAN, "output_kpl.txt", freeEph=.true.)
```

freeEph = .true. 表示保存后释放轨迹内存（节省内存）。文件格式为 ASCII，每行包含历元 + 6 维状态。

`readtrajectory`

从文件读取已有轨迹：

```
type(type_trajectory), allocatable :: traj(:)

call sat%readtrajectory(STATE_TYPE_CARTESIAN, "existing_traj.txt")
! 现在 sat%trajectory 和 sat%nEph 已填充
```

8.5 状态插值

`getStates` — 获取轨道根数

```
type(type_t) :: t_query
real(8) :: state(6)

call t_query%init(2025d0, 1d0, 1d0, 1d0, 15d0, 0d0)  ! 1h15m after epoch
state = sat%getStates(t_query)      ! 获取该时刻的轨道根数（类型与 sat%eleType 一致）
```

`getPV` — 获取位置速度

```
real(8) :: pv(6)

pv = sat%getPV(t_query)             ! 获取 J2000 系 Cartesian 位置速度
! pv(1:3) = r (归一化单位), pv(4:6) = v (归一化单位)
```

插值在已有的离散轨迹点上进行（需要先 `generateTrajectory` 或 `readtrajectory`）。

8.6 推力器

```
type(type_thruster) :: thr

! ... 配置 thr 参数 ...
call sat%addThruster(thr)    ! 附加推力器到卫星
```

推力器被集成到 `NumOP` 的力模型叠加中，在轨迹生成时自动生效。

8.7 `type_satelliteScenario` — 多星多站场景

```
type type_satelliteScenario
    type(type_timeSeries) :: ts          ! 时间序列
    type(type_satellite), allocatable :: satellites(:)  ! 卫星数组
    type(type_station), allocatable :: stations(:)      ! 地面站数组
    type(type_access), allocatable :: accessLinks(:)    ! 可见性链路
    ! ...
contains
    procedure :: init
    procedure :: generateObservations    ! 批量生成观测数据
    procedure :: reportAccess            ! 生成可见性报告
endtype
```

典型用法：

```
type(type_satelliteScenario) :: scenario

! 初始化场景
call scenario%init(ts)
! 添加卫星和地面站
! ...
! 生成所有可见弧段的观测数据
call scenario%generateObservations()
! 保存结果
call scenario%reportAccess("access_report.txt")
```

场景对象将卫星、地面站和可见性分析统一管理，是轨道确定前的准备步骤。详细用法见第15章 应用程序中的 `satelliteScenario` 程序。

8.8 卫星信息展示

```
call sat%show()
! 输出:
! === Satellite Information ===
! Number: 1
! Name: TestSat
! Mass: 500.0 kg
! Element Type: 1 (Keplerian)
! Force Model: K = 1 1 0 1 0 0 0 0
! Order: 8
! Propagator: numerical
! ============================
```

8.9 使用建议

1. **初始化后检查**：调用 `show()` 确认力模型配置和初始状态
2. **轨迹内存管理**：长弧段高采样率轨迹可能占用大量内存——使用 `freeEph=.true.` 在保存后释放
3. **状态类型一致性**：`eleType` 决定 `getStates` 的输出类型，与传播器内部使用的 NSG1 不同（内部自动转换）
4. **插值精度**：`getStates` / `getPV` 的插值精度取决于轨迹的时间分辨率——精细的时间步长产生更好的插值结果
5. **传播方法选择**：二体传播速度极快但不含摄动，仅适合轨道初步分析；数值传播精度高但计算量大
   \newpage

第9章 地面站

9.1 概述

地面站是卫星观测和通信的基础节点。Atlas 通过 `module_station` 提供了 `type_station` 对象，用于定义地面站的基本属性、计算站址在 ECF 和 J2000 系中的位置速度，以及生成地面站的惯

性系轨迹。

对应模块：`module_station`（Level 5）

9.2 `type_station` 地面站对象

```
type type_station
    integer :: number              ! 站编号
    character(100) :: name         ! 站名称
    real(8) :: longitude           ! 大地经度（rad），东经为正
    real(8) :: latitude            ! 大地纬度（rad），北纬为正
    real(8) :: altitude            ! 大地高（Ae，归一化单位）
    real(8) :: position(3)         ! ECF 直角坐标（Ae）
    type(type_timeSeries) :: ts    ! 关联时间序列
    real(8), allocatable :: trajectory(:, :)  ! J2000 系轨迹（6，nEpoch）
contains
    procedure :: init              ! 初始化
    procedure :: generateTrajectory ! 生成 J2000 轨迹
    procedure :: getPV             ! 获取某时刻 J2000 位置速度
    procedure :: show              ! 显示站信息
endtype
```

初始化

```
type(type_station) :: station

call station%init( &
    number = 1, &
    name = "Beijing", &
    longitude = 116.4d0, &    ! 东经 116.4 度
    latitude = 39.9d0, &      ! 北纬 39.9 度
    altitude = 50.0d0)        ! 海拔 50 米
```

初始化时，`init` 自动计算 ECF 直角坐标（使用 WGS-84 参考椭球，扁率 1/298.257223563，长半轴 6378.1363 km）。

9.3 大地坐标转换

`module_station` 包含大地坐标与直角坐标之间的转换子程序：

子程序	功能
`Geodetic`	大地坐标 (λ, φ, h) → ECF 直角坐标
`geo2pos`	封装 `Geodetic`，带单位处理
`pos2geo`	ECF 直角坐标 → 大地坐标

转换公式

使用 WGS-84 参考椭球参数：

$$
\begin{aligned}
N &= \frac{a_e}{\sqrt{\cos^2\varphi + (1-f)^2\sin^2\varphi}} \\
x &= (N+h)\cos\varphi\cos\lambda \\
y &= (N+h)\cos\varphi\sin\lambda \\
z &= \left[N(1-f)^2 + h\right]\sin\varphi
\end{aligned}
$$

其中 `a_e = 6378.1363 km`，`f = 1/298.257223563`。输出结果为归一化单位（以 `a_e` 为长度单位）。

9.4 生成地面站 J2000 轨迹

由于地球自转，地面站在惯性系（J2000）中的位置随时间变化。`generateTrajectory` 为时间序列中的每个历元计算地面站的 J2000 位置速度：

```
type(type_timeSeries) :: ts
type(type_t) :: t0

call t0%init(2025d0, 1d0, 1d0, 0d0, 0d0, 0d0)
call ts%init(t0, 60d0, 1440)   ! 1 天，60 秒间隔，1440 个点

call station%generateTrajectory(ts)
! station%trajectory 现在包含 1440 个历元的 (x,y,z,vx,vy,vz) in J2000
```

内部实现为：对每个历元，先通过 IAU1980 或 IAU2006A 计算旋转矩阵 HG，然后 `r_J2000 = HGᵀ · r_ECF`。

9.5 获取指定时刻的位置速度

```
type(type_t) :: t
real(8) :: pv(6)

call t%init(2025d0, 1d0, 1d0, 0d0, 5d0, 10d0)  ! 00:05:10
pv = station%getPV(t)
! pv(1:3): J2000 位置（归一化单位）
! pv(4:6): J2000 速度（归一化单位）
```

`getPV` 在已有 J2000 轨迹上进行时间插值获取指定时刻的位置和速度。

9.6 显示站信息

```
call station%show()
! 输出：站编号、名称、经度、纬度、高度、ECF 位置
```

9.7 在场景中使用地面站

`type_station` 常与 `type_timeSeries` 和 `type_satelliteScenario` 配合使用：

```
! 单站模式：独立创建和计算
type(type_station) :: myStation
type(type_timeSeries) :: ts

call myStation%init(number=1, name="Urumqi", &
    longitude=87.6d0, latitude=43.8d0, altitude=800d0)
call ts%init(t0, 60d0, 1440)
call myStation%generateTrajectory(ts)

! 多站模式：通过 type_satelliteScenario 管理
! 见第8章 卫星对象 中的 satelliteScenario 说明
```

9.8 使用建议

1. **坐标单位**：`longitude` 和 `latitude` 在存储时使用弧度，但 `init` 接受度输入（自动转换为弧度）
2. **轨迹生成**：地面站 J2000 轨迹需要 EOP 数据（极移、DUT1、章动系数）用于坐标变换——确保 `data/data_op/` 下有 `Nutation.dat` 和 EOP 表
3. **轨迹内存**：`trajectory(:, :)` 的大小为 `(6, nEpoch)`，高频率长时间弧段会占用较多内存——按需设置 `type_timeSeries` 的采样间隔
4. **可见性分析**：地面站的可见性分析由 `module_access` 负责，详见第10章 可见性与观测类型 \newpage

第10章 可见性与观测类型

10.1 概述

可见性分析是卫星任务规划和观测仿真的核心环节。Atlas 的 `module_access` 提供卫星与地面站之间的可见性判断、观测数据生成和链路管理能力。多种观测类型（赤经赤纬、方位仰角、测距、多普勒、GNSS 等）可通过统一的接口配置。

对应模块：`module_access`（Level 5），`module_atmosphere`（Level 4，对流层折射）

10.2 观测类型

Atlas 定义了以下观测类型（`module_access` 中的常量）：

常量	值	类型	观测量	维数	典型用途
`ALPHA_DELTA`	11	赤经/赤纬	(α, δ)	2	光学天体测量、空间监视
`RADAR`	12	雷达	(ρ, α, δ) 或 (ρ, A, E)	3	雷达跟踪
`RANGING`	13	测距	(ρ)	1	卫星激光测距 (SLR)
`GNSS`	14	导航	(x,y,z,vx,vy,vz)	6	GPS/北斗精密定轨
`POSITION`	15	位置	(x, y, z)	3	全位置观测
`AZIMUTH_ELEVATION`	16	方位/仰角	(A, E)	2	地基光学/雷达
`RANGE_DOPPLER`	17	距离+多普勒	(ρ, ρ̇)	2	DORIS 等跟踪系统

10.3 观测几何

赤经/赤纬 (α, δ)

从地面站指向卫星的单位矢量在 J2000 天球上的投影：

```
ρ_vec = r_sat - r_station    (J2000 系)
α = atan2(ρ_y, ρ_x)          (赤经, 0 to 2π)
δ = asin(ρ_z/|ρ|)            (赤纬, -π/2 to π/2)
```

方位/仰角 (A, E)

在站心地平坐标系中：

```
E = asin(r_horiz_up / |r_horiz|)
A = atan2(r_horiz_east, r_horiz_north)
```

卫星可见的基本判据：`E > E_min`（通常 E_min 设置为 5°–10°）。

测距 (ρ)

简单欧氏距离：

$$\rho = \left|\mathbf{r}_{\text{sat}} - \mathbf{r}_{\text{station}}\right|$$

高精度应用需考虑光行时修正（当前接口为瞬时几何距离）。

多普勒 (ρ̇)

距离变率（通过速度投影）：

$$\dot{\rho} = \left(\mathbf{v}_{\text{sat}} - \mathbf{v}_{\text{station}}\right) \cdot \hat{\boldsymbol{\rho}}$$

其中 `ρ`为测站到卫星的单位矢量。

10.4 可见性约束

`type_access` 支持以下约束条件：

约束	说明
最小/最大仰角	观测仰角必须在 `[E_min, E_max]` 范围内
太阳距离	卫星-太阳最小角距离（避免日盲）
月球距离	卫星-月球最小角距离
地影遮挡	卫星是否在地球阴影中
月影遮挡	卫星是否在月球阴影中

所有约束在可见性计算时自动检测，不满足约束的时段被排除。

10.5 对流层折射修正

`module_atmosphere` 中包含对流层折射改正，用于修正低仰角观测：

```
subroutine tropospheric_refraction(Elev, T, fh, Elev1)
! 输入:
!   Elev  - 原始观测仰角 (rad)
!   T     - 温度 (K), 默认 293.15
!   fh    - 相对湿度 (0-1), 默认 0.7
! 输出:
!   Elev1 - 折射修正后的仰角 (rad)
```

折射修正量：0.01–0.03 rad（~0.0002–0.0005 度）。仅低仰角（< 10°）时显著，高仰角可忽略。

10.6 `type_access` 基本使用

```
type type_access
    ! source/target: 卫星 ↔ 地面站 的 link
    ! observationType: 观测类型常量
    ! constraints: 仰角范围、天体距离限制、地影/月影
endtype
```

可见性分析通常集成在 `type_satelliteScenario` 中使用（见第8章 卫星对象），通过 `generateObservations` 和 `reportAccess` 方法批量处理，无需直接操作 `type_access`。

10.7 观测残差与定轨

观测数据生成后，观测残差（O − C）的计算由 `module_residual` 负责，批处理最小二乘和卡尔曼滤波由 `module_SLE` 和 `module_ODor` 完成。详细内容见第11章 轨道确定。

10.8 使用建议

1. **观测类型选择**：光学观测 → `AZIMUTH_ELEVATION`；雷达 → `RADAR`；SLR → `RANGING`；GNSS 精密定轨 → `GNSS`
2. **仰角约束**：设置合理的 `E_min`（通常 5°–10°），过低的仰角导致过大的对流层折射误差
3. **观测权重**：不同观测类型有不同的测量精度，典型 σ 值：角度 1–5 角秒，测距 1–10 m，GNSS 伪距 0.5–2 m

4. **光行时**：当前测距计算为瞬时几何距离，高精度应用（< 1 cm）需考虑光行时修正
5. **对流层修正**：低仰角（< 10°）时折射修正显著，建议开启
   \newpage

第11章 轨道确定

11.1 概述

轨道确定（Orbit Determination, OD）是利用观测数据反演卫星轨道状态的过程，是卫星测控和空间态势感知的核心技术。Atlas 提供了一套完整的轨道确定工具链，涵盖：

- **观测残差计算**（`module_residual`）：O−C 计算、异常值检测、RMS 统计
- **线性方程组求解**（`module_SLE`）：法方程构建、批处理解算、卡尔曼滤波数据融合
- **初轨确定 (IOD)**：从有限观测数据确定初步轨道
- **精密定轨 (POD)**：基于大量观测数据的高精度轨道计算
- **三种滤波方法**：批处理最小二乘、扩展卡尔曼滤波、序贯滤波
- **灵活的参数估计**：轨道状态、阻力系数、光压系数等

观测类型的定义和可见性分析由 `module_access` 负责，详见第10章 可见性与观测类型。

对应模块：`module_IODor`（Level 5），`module_ODor`（Level 5），`module_residual`（Level 5），`module_SLE`（Level 3），`module_STM`（状态转移矩阵）

11.2 观测残差：`module_residual`

观测残差（O − C，即观测值减去计算值）是构建代价函数和迭代求解的基础。

```
type type_residual
    ! 存储观测值与计算值之差 (O - C)
    ! 支持异常值检测和 RMS 统计
endtype

type type_acc
    ! 精度评估类型，存储 RMS 和统计量
endtype
```

主要功能

- **残差向量计算**：对每个观测历元计算 `O - C`
- **异常值（outlier）检测**：基于残差统计特性自动识别和标记异常观测值
- **加权 RMS 计算**：`RMS = √(Σ w_i·(O-C)_i² / Σ w_i)`

残差是 Gauss-Newton 迭代的驱动力：

$$\Delta \mathbf{z} = \mathbf{z}_{\text{obs}} - h(\mathbf{x}_{\text{ref}})$$

其中 `h(x_ref)` 为基于参考轨道的计算观测量。

11.3 线性方程组：`module_SLE`

`module_SLE`（System of Linear Equations）是批处理最小二乘和卡尔曼滤波的数值核心：

```
type type_SLE
    ! BTB: 法方程系数矩阵 (HᵀWH)
    ! BTY: 法方程右端项 (HᵀW·(O-C))
    ! X: 解向量 Δx̂
    ! P: 协方差矩阵
contains
    procedure :: initialize    ! 初始化为零矩阵
    procedure :: addData       ! 累加一组观测方程
    procedure :: solve         ! 解算法方程
    procedure :: propagate     ! 通过 STM 传播状态和协方差
    procedure :: dataFusion    ! 卡尔曼滤波测量更新
endtype
```

批处理最小二乘中的使用

```
SLE%initialize(nObs, nPar)        ! 初始化法方程为零矩阵
For each observation:
    SLE%addData(H_i, y_i, W_i)    ! 累加 HᵢᵀWᵢHᵢ 和 HᵢᵀWᵢyᵢ
SLE%solve(X, P)                    ! 解算 X = (HᵀWH)⁻¹HᵀWy
```

卡尔曼滤波中的使用

```
call SLE%dataFusion(H, y, W, X, P)    ! EKF 测量更新
```

结合 `propagate` 方法实现卡尔曼滤波递推：

```
时间更新:   X̂_k⁻ = propagate(X̂_{k-1}⁺)
           P_k⁻ = Φ·P_{k-1}⁺·Φᵀ + Q
测量更新:   K = P⁻·Hᵀ(H·P⁻·Hᵀ + R)⁻¹
           X̂⁺ = X̂⁻ + K·(z − h(X̂⁻))
           P⁺ = (I − K·H)·P⁻
```

观测方程与偏导数

对于第 i 个观测，线性化的观测方程为：

$$y_i = H_i \cdot \Delta\mathbf{x} + \varepsilon_i$$

其中：

- `y_i = O_i − C_i` 为观测残差
- `H_i = ∂h_i/∂x` 为偏导数矩阵（由 `module_STM` 解析计算或数值差分）
- `W_i = 1/σ_i²` 为观测权重（σ_i 为观测噪声标准差）

`SLE%addData` 的累加过程：

$$\mathbf{H}^T\mathbf{W}\mathbf{H} \leftarrow \mathbf{H}^T\mathbf{W}\mathbf{H} + H_i^T W_i H_i$$

$$\mathbf{H}^T\mathbf{W}\mathbf{y} \leftarrow \mathbf{H}^T\mathbf{W}\mathbf{y} + H_i^T W_i y_i$$

> **命名说明**： `module_SLE` 的缩略词与"Satellite Laser Ranging"相同，但该模块是通用的线性

> 方程组求解器（"System of Linear Equations"）。SLR 观测数据通过 `RANGING` 类型的观测模型处理。

11.4 初轨确定：`type_IODor`

`module_IODor` 实现了从角度/测距数据确定初始轨道的经典方法。

```
type type_IODor
    ! IOD 引擎，使用 General Laplace 方法
    ! 支持配置最大数据长度
contains
    procedure :: init         ! 初始化
    procedure :: solve        ! 执行 IOD 解算
    procedure :: show         ! 显示结果
    procedure :: saveIODor    ! 保存结果
endtype
```

方法原理

初轨确定基于**广义 Laplace 方法**，适用于角度观测（赤经/赤纬）和/或测距数据。

对于角度观测 `(α, δ)`，Laplace 方法利用跟踪弧段中多次观测的几何关系建立方程组，通过迭代求解卫星的距离及其变化率。

使用示例

```
type(type_IODor) :: iod
real(8) :: observations(:,:), initialState(6)

call iod%init(maxDataLength=100)
! 添加观测数据和测站信息 ...
call iod%solve(initialState)
! initialState 为 J2000 系中的 Cartesian 状态（归一化单位）
call iod%show()
call iod%saveIODor("iod_result.txt")
```

11.5 精密定轨：`type_ODor`

`type_ODor` 是 Atlas 中最大的模块（~77 KB），实现了完整的精密轨道确定引擎。

```
type type_ODor
    ! 精密定轨引擎
    ! method = 1: 批处理最小二乘 (Batch Least Squares)
    ! method = 2: 扩展卡尔曼滤波 (EKF)
    ! method = 3: 序贯滤波 (Sequential Filter)
contains
    procedure :: init          ! 初始化
    procedure :: solve         ! 批处理解算（method=1）
    procedure :: invoke        ! EKF 更新（method=2）
    procedure :: saveEph       ! 保存星历
    procedure :: SS2ODor       ! 场景 → ODor 数据转移
    procedure :: ODor2SS       ! ODor → 场景数据转移
endtype
```

三种滤波方法

方法	`method`	说明	适用场景
批处理最小二乘	1	一次性处理全部弧段数据，迭代求解	事后精密定轨（POD）
扩展卡尔曼滤波 (EKF)	2	逐观测更新，实时递推估计	实时轨道确定
序贯滤波	3	分弧段批处理，弧段间递推	多弧段联合定轨

批处理最小二乘原理

给定观测值 `z` 和观测模型 `h(x)`，最小化加权残差平方和：

$$J(\mathbf{x}) = (\mathbf{z} - h(\mathbf{x}))^T \mathbf{W} (\mathbf{z} - h(\mathbf{x})) \rightarrow \min$$

通过 Gauss-Newton 迭代求解：

1. **线性化**：`Δz = H · Δx`，其中 `H = ∂h/∂x` 由 STM 提供

2. **构建法方程**：通过 `SLE%addData` 累加 H^TWH 和 $H^TW\Delta z$
3. **解算法方程**：`SLE%solve` → $\Delta\hat{x} = (H^TWH)^{-1}H^TW\Delta z$
4. **状态更新**：$\hat{x} \leftarrow \hat{x} + \Delta x$
5. **迭代收敛**：直到 $||\Delta x|| < \varepsilon$

11.6 待估参数组：`type_varGroup`

`module_ODor` 通过 `type_varGroup` 定义哪些参数参与估计：

`varGroup` ID	待估参数	维数	说明
1	轨道状态	6	仅 6 维轨道根数
2	轨道状态 + 阻力系数 Cd	7	同时估计大气阻力
3	轨道状态 + 光压系数	7	同时估计 SRP
4	轨道状态 + Cd + 光压	8	同时估计阻力和光压
5	轨道状态 + ECOM 9 参数	15	完整 ECOM 光压参数估计
11	某对象的钟差 bias0	1	钟差多项式的前 1 个系数
12	某对象的钟差 bias0 + drift	2	前 2 个系数
13	某对象的钟差 bias0 + drift + drift2	3	全部 3 个系数

钟差组（label 11/12/13）只含钟差参数，不含轨道状态量；`nVar` = 1/2/3 决定估计 $bias(t) = bias0 + drift\cdot dt + drift2\cdot dt^2$（dt 以秒计）的**前 nVar 个系数**，基准历元是 `type_ODor%t`。`type_varGroup%objType` 指明参数属于卫星还是地面站（`OBJECT_TYPE_SATELLITE` / `OBJECT_TYPE_STATION`），`origin` 是该类型下的编号。

钟差只出现在**测距（RANGING）**观测模型里（$rho_obs = rho + (clk_R - clk_T)\cdot c$），其它观测量建钟差组会得到全 0 列，初始化时即报错。注意秩亏：测距只能观测到钟差之差，所有钟同加一个常数不可观测，**必须由应用固定一个参考钟**（典型做法是地面站钟固定为 0），库本身不加任何约束。钟差组目前只支持批处理方法。

参数估计使用了 `module_STM` 提供的解析状态转移矩阵（包含 J2、第三体和 SRP 的偏导数），在 NSG2 非奇异根数下计算。

11.7 状态转移矩阵（STM）

`module_STM`（独立模块）提供了解析的状态转移矩阵计算：

```
! STM: d(x_t)/d(x_0) — 6×6 或 7×7 矩阵
! 包括对 J2、日月第三体和 SRP 参数的偏导数
! 基于 NSG2 非奇异根数解析公式
```

主要功能：

- **nVar = 6**：轨道状态偏导数
- **nVar = 7**：轨道状态 + SRP 偏导数
- 用于精密定轨中的 H 矩阵构建和协方差传播

STM 在定轨迭代中计算观测方程 `H = ∂h/∂x · d(x_t)/d(x_0)` 的偏导数链。

11.8 定轨处理流程

批处理 POD 典型流程

```
1. 准备观测数据和初始轨道猜测 x̂₀
2. 创建 ODor 对象，设置 method=1, varGroup
3. call ODor%init(...)
4. call ODor%solve(maxIter=10, convergence=1e-8)
   ┌──────────────────────────────────────┐
   │ For each iteration:                  │
   │   a. 用当前 x̂ 传播参考轨道            │
   │   b. 对每个观测计算 O-C 和 H 矩阵      │
   │   c. SLE%addData 构建法方程            │
   │   d. SLE%solve → Δx̂                   │
   │   e. x̂ ← x̂ + Δx̂                      │
   │   f. 检查收敛                         │
   └──────────────────────────────────────┘
5. 收敛后保存精密星历和协方差
```

EKF 实时处理流程

```
1. 初始化状态和协方差 (x̂₀, P₀)
2. For each new observation:
   a. 时间更新: SLE%propagate → X̂⁻ = Φ·X̂⁺
                                P⁻ = Φ·P⁺·Φᵀ + Q
   b. 测量更新: SLE%dataFusion → X̂⁺ = X̂⁻ + K·(z − h(X̂⁻))
                                  P⁺ = (I − K·H)·P⁻
3. Output: filtered state x̂ 和 covariance P
```

11.9 数据接口

`type_ODor` 与场景对象之间可以通过方法进行数据转移：

```
call ODor%SS2ODor(scenario)    ! 从卫星场景导入定轨设置
call ODor%ODor2SS(scenario)    ! 将定轨结果导出到卫星场景
```

这允许用户在 `satelliteScenario` 程序中配置多星多站环境，然后无缝接入精密定轨。

11.10 使用建议

1. **初轨精度**：IOD 提供的是近似轨道，通常作为 POD 的初始猜测。对于角度观测，单站单弧段 IOD 的距离误差可能较大
2. **收敛性**：批处理最小二乘的收敛性依赖于初始猜测的质量——如果 IOD 结果不佳，POD 可能发散
3. **观测权重**：不同观测类型设置合理的先验 σ 值——典型设置为：角度 1–5 角秒，测距 1–10 m，GNSS 伪距 0.5–2 m
4. **异常值剔除**：在批处理收敛后检查残差，剔除超过 3σ 的异常观测后重新解算
5. **法方程条件数**：参数间的高相关性（如轨道面内参数与阻力系数）可能导致法方程病态——检查协方差矩阵对角元素和相关系数
6. **参数选择**：LEO 卫星通常需要估计阻力系数（`varGroup=2`）；GNSS 卫星通常只需估计光压（`varGroup=3`）；ECOM 模型需要足够长的弧段才能可靠估计
7. **STM 使用**：解析 STM 效率远高于数值有限差分，但仅适用于 NSG2 根数——在 IOD/Cartesian 模式下需要数值偏导数

\newpage

第12章 三体问题

12.1 概述

圆形限制性三体问题（CRTBP, Circular Restricted Three-Body Problem）是研究航天器在两个大天体引力场中运动的基础模型。Atlas 提供了一套完整的 CRTBP 分析工具，包括：

- CRTBP 系统定义（地月系、日地系等）
- 拉格朗日平动点位置计算
- Halo 轨道三阶解析设计
- Lissajous 轨道设计
- Lambert 问题求解

对应模块：`module_CRTBP`（Level 1）、`module_halo`（Level 2）、`module_LissajousOrbit`（Level 2）、`module_lambert`（Level 1）

> **参考文档**：`data/doc/` 目录下有详细的 Halo 轨道和 Lissajous 轨道算法文档（中文），包含完整的数学推导。

12.2 CRTBP 系统：`type_CRTBP`

系统定义

CRTBP 描述质量为无穷小的航天器在两个大天体（主天体 m_1 和次天体 m_2）引力场中运动的问题。两主天体绕公共质心做圆轨道运动。运动方程在旋转坐标系中表示。

```
type type_CRTBP
    character(50) :: name          ! 系统名称（如 "Earth-Moon"）
    real(8) :: m1, m2              ! 主天体质量 (kg)
    real(8) :: a12                 ! 两主天体距离 (km)
    real(8) :: TU                  ! 时间单位 (s)，由 Kepler 第三定律导出
    real(8) :: mu                  ! 质量参数 = m2/(m1 + m2)
    real(8) :: L1(2), L2(2), L3(2), L4(2), L5(2)  ! 五个平动点坐标
contains
    procedure :: init
    procedure :: getLagrangePointCoord     ! 获取平动点坐标
    procedure :: getLagrangePointParams    ! 获取平动点特征参数
endtype
```

初始化

```
type(type_CRTBP) :: em

! 地月系 CRTBP
call em%init("Earth-Moon", &
    m1_in = 5.97219d24, &    ! 地球质量 (kg)
    m2_in = 7.34767309d22, &  ! 月球质量 (kg)
    a12_in = 384400d0)        ! 地月距离 (km)
```

初始化自动计算 `TU` 和 `mu`，并解算五个拉格朗日平动点的旋转坐标。

拉格朗日平动点

平动点的坐标存储在旋转坐标系中（质心为原点，x 轴从 m_1 指向 m_2）：

平动点	类型	位置
L1	共线	m_1 和 m_2 之间
L2	共线	m_2 外侧
L3	共线	m_1 外侧（与 m_2 相反方向）
L4	三角	m_1-m_2 连线前方 60°

平动点	类型	位置
L5	三角	m_1-m_2 连线后方 60°

平动点值在 `init` 时通过解 Euler 五次方程（共线点）和解析公式（三角点）计算得到。

拉格朗日点标识符

```
integer, parameter :: LAGRANGE_L1 = 1
integer, parameter :: LAGRANGE_L2 = 2
integer, parameter :: LAGRANGE_L3 = 3
integer, parameter :: LAGRANGE_L4 = 4
integer, parameter :: LAGRANGE_L5 = 5
```

使用平动点坐标

```
real(8) :: coord(2)
coord = em%getLagrangePointCoord(LAGRANGE_L1)
! coord(1) = x (沿 m1-m2 连线), coord(2) = y
```

单位体系

CRTBP 系统使用自身的归一化单位：

物理量	归一化单位	取值
长度	a12	两主天体距离
时间	TU = $2\pi\sqrt{(a12^3/G(m1+m2))}$	主天体轨道周期/2π
质量	m1+m2	总质量

12.3 Lambert 问题求解：`module_lambert`

Lambert 问题：给定两个位置矢量 r_1, r_2 和飞行时间 Δt，求解连接二者的开普勒轨道。

```
! module_lambert 中的求解器
! 从两个位置矢量和时间差求解 Keplerian 轨道根数
```

该模块用于深空探测的转移轨道设计和初轨确定。

12.4 Halo 轨道：`type_halo`

Halo 轨道是围绕 CRTBP 共线平动点（L1, L2, L3）的三维周期轨道。`module_halo` 实现了三阶解析近似：

```
type type_halo
    real(8) :: c2, c3, c4          ! Legendre 展开系数
    real(8) :: omega0              ! 基频
    real(8) :: k                   ! 形状参数
    ! 二阶系数 a21-a24, b21, b22, d21
contains
    procedure :: init              ! 初始化
    procedure :: compute_state     ! 计算给定时间的状态
endtype
```

三阶解析解由 Richardson (1980) 的理论导出。具体算法细节见 `data/doc/halo_orbit.md`。

使用方法

```
type(type_halo) :: halo
type(type_CRTBP) :: em

! 初始化 CRTBP 系统
call em%init("Earth-Moon", ...)

! 初始化 Halo 轨道（绕 L1 点，北向）
call halo%init(em, LAGRANGE_L1, north=.true., Az=5000d0)
! Az = 5000 km 为 z 方向振幅

! 计算某时刻的轨道状态
real(8) :: state(6)
state = halo%compute_state(t)
```

12.5 Lissajous 轨道：`type_LissajousOrbit`

Lissajous 轨道是比 Halo 轨道更一般的平动点拟周期轨道，在 xy 平面和 z 方向有不同的振幅和频率：

```
type type_LissajousOrbit
    ! 参数: A_xy (面内振幅), A_z (面外振幅)
    !       phi_xy, phi_z (初始相位)
contains
    procedure :: init
    procedure :: compute_state
endtype
```

Lissajous 轨道基于线性化 CRTBP 解（面内 / 面外解耦的简谐运动），适合作为数值延拓（differential correction）的初始猜测。

具体算法细节见 `data/doc/Lissajous_orbit_design_algorithum.md` 和 `data/doc/深空探测器轨道力学_Lissajous.md`。

使用方法

```
type(type_LissajousOrbit) :: liss
type(type_CRTBP) :: em

call liss%init(em, LAGRANGE_L2, A_xy=30000d0, A_z=50000d0, &
               phi_xy=0d0, phi_z=90d0)
real(8) :: state(6)
state = liss%compute_state(t)
```

12.6 使用建议

1. **CRTBP 单位独立性**：CRTBP 模块使用自身的归一化单位制，与地球卫星的归一化单位不同。在 CRTBP 与 Earth-centered 计算之间切换时务必注意单位转换
2. **解析解的适用范围**：Halo 轨道的三阶解析解在中等振幅（Az < ~10000 km 地月系）时精度良好，大幅值情况需要数值修正
3. **Lambert 求解**：Lambert 求解器适用于行星际转移轨道设计的初步分析——高精度任务建议使用数值方法验证
4. **参考文档**：`data/doc/` 下有详细的中文算法推导，是理解内部实现的重要参考
\newpage

第13章 行星历表

13.1 概述

行星历表（Planetary Ephemeris）提供太阳、月球和各大行星在任意时刻的高精度位置和速度。Atlas 通过 `module_planets` 支持三种历表模式，主要依赖 JPL DE440 数值历表。底层历表文件读取由 legacy `PLEPH.for` 实现。

对应模块：`module_planets`（Level 2），`PLEPH.for`（Legacy）

13.2 三种历表模式

`module_planets` 根据 `planetsEph` 全局变量选择历表模式：

模式	说明	适用场景
JPL DE440	高精度数值历表	精密定轨、高精度预报
解析历表	近似解析公式	快速计算、教育用途
CRTBP	Earth-Moon 三体模型	地月空间轨道设计

13.3 天体标识

`module_planets` 定义的天体常数：

标识符常量	说明	备注
`nMercury`	水星	ID 1
`nVenus`	金星	ID 2
`nEarth`	地球	ID 3（中心天体）
`nMars`	火星	ID 4
`nJupiter`	木星	ID 5
`nSaturn`	土星	ID 6
`nUranus`	天王星	ID 7
`nPluto`	冥王星	ID 9
`nMoon`	月球	ID 10
`nSun`	太阳	ID 11
`nSSB`	太阳系质心	ID 12

所有天体的质量比（相对于地球）和半径数据也定义在 `module_planets` 中。

13.4 行星历表接口

`planetsPV` — 获取行星位置速度

```
subroutine planetsPV(TT, center, body, PV)
! 输入:
!   TT     - 地球力学时 (MJD)
!   center - 中心天体 ID
!   body   - 目标天体 ID
! 输出:
!   PV(6)  - 位置 (1:3) 和速度 (4:6)，单位：km, km/s
```

这是获取天体位置速度的核心接口。例如，获取太阳相对于地球的位置：

```
real(8) :: TT, solarPV(6)

call UTC2TT(UTC, TT)                        ! UTC → TT
call planetsPV(TT, nEarth, nSun, solarPV) ! Earth → Sun 矢量
! solarPV(1:3) = 太阳在 J2000 系中的位置 (km)
! solarPV(4:6) = 太阳速度 (km/s)
```

常用调用模式

```
! 月球位置（地心）
call planetsPV(TT, nEarth, nMoon, LunarPV)

! 太阳位置（地心）
call planetsPV(TT, nEarth, nSun, SolarPV)

! 木星位置（地心，用于第三体摄动）
call planetsPV(TT, nEarth, nJupiter, JupiterPV)

! 地球位置（日心）
call planetsPV(TT, nSun, nEarth, EarthPV)
```

13.5 JPL DE440 历表

JPL DE440（Development Ephemeris 440）是 JPL 发布的最新高精度行星历表，覆盖约 1550–2650 年，精度优于 DE430。

文件格式

JPL 历表以二进制格式存储，包含各天体的 Chebyshev 多项式系数。`PLEPH.for`（CALTECH/JPL 原始 Fortran 77 代码）负责读取和插值。

历表文件位置

历表文件位于 `data/data_op/` 目录。文件路径在 `module_planets` 中根据操作系统选择：

- **Linux**：`data/data_op/lnx*`
- **Windows**：`data/data_op/` 或指定路径

PLEPH 接口

```
subroutine PLEPH(TT, NTARG, NCENT, PV)
! TT    - 儒略日 (TT)
! NTARG - 目标天体 ID（JPL 编号）
! NCENT - 中心天体 ID（JPL 编号）
! PV(6) - 位置速度 [km, km/s]
```

`PLEPH` 使用 Chebyshev 多项式插值，以 32 天为间隔的高精度 Chebyshev 系数压缩数据。

13.6 第三体摄动中的使用

`module_perturbationThirdBody` 大量调用 `planetsPV`。在每个积分步的右端函数中，根据力模型开关获取所需天体的位置速度：

```
! 月球引力摄动 (K2 = 1)
if (Moon == 1) then
    call planetsPV(TT, nEarth, nMoon, LunarPV)
    r_lunar = LunarPV(1:3)/length_unit
    F_moon = -mass_lunar * (delta/delta_norm^3 + r/r_norm^3)
end if
```

13.7 解析历表

作为 JPL 历表的备选，`module_planets` 实现了简化的解析历表。解析历表精度较低（~100 km 量级），但计算速度快，不依赖外部数据文件，适合教育用途和快速原型。

解析历表的选择由模块内部的 `planetsEph` 变量控制。

13.8 使用建议

1. **历表数据**：确保 JDE440 历表文件在正确的路径下——`module_planets` 初始化时会检查文件存在性
2. **时间系统**：`planetsPV` 和 `PLEPH` 要求 TT（地球力学时）——务必先用 `UTC2TT` 转换
3. **速度输出**：`PV(4:6)` 的速度矢量与 `PV(1:3)` 的位置矢量在同一坐标系（ICRS/J2000）中
4. **性能**：JPL 历表的 Chebyshev 插值开销很小（32 天间隔意味着 1 次查表可服务多步积分），是第三体摄动计算的主要开销
   \newpage

第14章 姿态模型

14.1 概述

航天器姿态描述了星体坐标系相对于参考坐标系的方向。Atlas 的 `module_attitude` 提供了一套姿态描述和变换工具，支持多种指向模式（对地定向、惯性固定、太阳定向）以及四元数、欧拉角、方向余弦矩阵等多种表示方式。

对应模块：`module_attitude`（Level 3）

14.2 坐标框架

`module_attitude` 定义了以下坐标框架：

框架	说明
J2000	地心天球参考系（惯性系）
ECF	地心地固系
COCS	卫星质心轨道坐标系（Centroid Orbit Coordinate System）
SBCS	卫星本体坐标系（Spacecraft Body Coordinate System）

COCS（质心轨道坐标系）

COCS 是一种常用的对地定向参考系：

- **z 轴**：指向地心（nadir）
- **y 轴**：轨道面法向的反方向
- **x 轴**：沿飞行方向（在轨道面内，与 z 轴正交）

SBCS（星体坐标系）

SBCS 是固定于卫星结构的坐标系，其相对 COCS 或 J2000 的旋转即卫星姿态。

14.3 姿态表示

`module_attitude` 支持多种姿态表示方式，通过 `module_rotator` 提供底层转换：

表示方式	变量数	特点
四元数 (Quaternion)	4	无奇异性，适合数值积分
欧拉角 (Euler Angles)	3	直观，但有万向节锁定问题
方向余弦矩阵 (DCM)	9 (3×3)	线性变换，冗余表示
YPR (Yaw-Pitch-Roll)	3	航空航天常用约定

四种表示之间可以通过 `module_rotator` 的函数互相转换。

旋转序列

`module_rotator` 支持以下欧拉角旋转序列：

- **313 (zxz)**：经典 Euler 角序列
- **312 (zxy)**：常用于轨道动力学
- **321 (zyx)**：Yaw-Pitch-Roll 航空标准序列

14.4 指向模式

`module_attitude` 支持以下姿态指向模式：

地球定向（Nadir Pointing）

这是对地观测卫星最常用的姿态模式——卫星始终将星体 z 轴（通常安装有效载荷）对准地心：

```
Z_sbcs → -r_sat (nadir方向)
Y_sbcs → -h (轨道角动量反方向)
X_sbcs → Y × Z (完成右手系)
```

惯性固定（Inertial Fixed）

星体相对于 J2000 惯性系保持固定方向。适用于天文观测和深空探测器的巡航阶段。

太阳定向（Sun Pointing）

星体指定轴对准太阳方向。常用于太阳能帆板定向和日冕观测。

14.5 姿态变换

坐标变换方法

```
! 预测某时刻的姿态
call att%predict(t)

! 计算姿态矩阵
call att%calAtti()

! 坐标矢量旋转变换
call att%coorTrans(v_in, v_out, from_frame, to_frame)
```

使用示例

```
type(type_attitude) :: att
type(type_t) :: t

! 初始化对地定向姿态
call att%init(attitude_mode="nadir")

! 在指定时刻计算姿态
call t%init(2025d0, 1d0, 1d0, 0d0, 0d0, 0d0)
call att%predict(t)

! 矢量变换：ECF → SBCS
real(8) :: v_ecf(3), v_sbcs(3)
v_ecf = (/1d0, 0d0, 0d0/)
call att%coorTrans(v_ecf, v_sbcs, "ECF", "SBCS")
```

14.6 与轨道计算的集成

姿态信息在以下场景中与轨道计算耦合：

1. **SRP 计算**：非球形卫星的 `amrSRP` 随姿态变化（ECOM 模型通过经验参数部分补偿）
2. **推力方向**：`module_thruster` 支持在 SBCS 中定义推力方向，姿态决定了推力的惯性方向

3. **观测仿真**：卫星本体上传感器的指向由姿态决定

14.7 底层转换：`module_rotator`

`module_rotator`（Level 0）提供四元数、欧拉角、DCM 之间的基础转换函数。更多细节见附录模块参考。

14.8 使用建议

1. **姿态传播**：当前姿态模型为运动学描述（不包含力矩和角速度动力学）——适合短期姿态分析，长期姿态传播需加入姿态动力学
2. **对地定向**：大多数 LEO 对地观测卫星使用对地定向模式——这是默认选项
3. **四元数积分**：如需长时间姿态传播，建议使用四元数表示以避免奇异性
   \newpage

第15章 应用程序

15.1 概述

Atlas 提供了 15 个独立的命令行应用程序（`app/` 目录），每个程序都是自包含的 Fortran `program` 单元，直接调用库模块完成特定任务。

所有应用程序编译后的可执行文件位于 `bin/` 目录，通过命令行参数配置。

文件路径：`app/*.f90`

15.2 核心应用程序

`app_orbitPrediction` — 数值轨道预报

最全面的轨道预报程序，支持完整的力模型配置。

命令行参数（21 个）：

参数	说明	单位
起始时间	年 月 日 时 分 秒	—
终止时间	年 月 日 时 分 秒	—
初始 Keplerian 根数	a, e, i, Ω, ω, M	km, —, deg
步长	输出采样步长	秒
输出文件名	轨迹保存路径	—
历表类型	(JPL/解析)	—
Cd, amrDrag, F107, Ap	大气阻力参数	—
reflectivity, amrSRP	光压参数	—
order	引力场阶次	—
K1–K8	8 个力模型开关	0/1

参考文档：`release/readme_orbitPrediction.md` 有完整的参数说明和示例。

`app_observation` — 观测仿真

仿真地面站对卫星的观测数据（方位/仰角、赤经/赤纬、距离/距离率）。

输入：卫星星历文件 + 地面站参数 + 时间设置（13 个参数）
输出：可见弧段的观测数据文件

参考文档：`release/readme_observation.md`

`app_initialOrbitDetermination` — 初轨确定

从角度/测距数据使用 General Laplace 方法确定初轨。

观测类型：AH（方位+仰角）、AD（赤经+赤纬）、RRD（距离+距离率）

参考文档：`release/readme_initialOrbitDetermination.md`

`app_orbitDetermination` — 精密定轨

基于卫星场景配置文件的批处理精密轨道确定。

输入：`data/input/` 下的场景文本文件
方法：批处理最小二乘

15.3 场景与仿真程序

`satelliteScenario` — 多星多站场景

完整的卫星场景模拟器，从配置文件读取多颗卫星和多个地面站的定义，生成观测数据和可见性报告。

功能：

- 自动生成所有卫星-地面站的可见弧段
- 计算并输出各类型观测数据
- 支持多功能配置（观测类型、仰角约束等）

`satellite` — 单星轨道计算

简化的单星轨迹计算程序，适合快速轨道预览。

`satellite_DRO` — 远距逆行轨道

针对远距逆行轨道（Distant Retrograde Orbit）的专用计算程序，包含地月空间 CR3BP 中的 DRO 设计和传播。

`prediction_thruster` — 带推力的轨道预报

在轨道预报中加入推力/机动模型，适合轨道转移和站位保持仿真。

15.4 批处理程序

`batch_GNSS` — GNSS 批量定轨

使用 GNSS 精密位置/速度观测进行批量轨道确定。配置直接在源码中修改（非命令行参数）。

`batch_position_beidou` — 北斗批量定位

处理北斗卫星观测数据的定位程序。支持批量处理多个历元的北斗卫星数据。

`batch_AzimuthElevation` — 方位/仰角批量处理

批量计算和分析方位角/仰角观测数据。

`ekf_angleData` — 角度数据 EKF 滤波

使用扩展卡尔曼滤波器处理仅角度（方位/仰角）观测数据。

15.5 演示与测试程序

`halo_orbit` — Halo 轨道计算

计算并输出 CR3BP 中 Halo 轨道的三阶解析解数据。

`cartesian` — 坐标变换演示

`type_cartesian` 对象的坐标变换演示。

`time` — 时间系统演示

时间系统创建和转换的演示。

15.6 程序与模块的对应关系

程序	主要使用的模块
`app_orbitPrediction`	`module_satellite`, `module_orbitPredictionNumerical`
`app_observation`	`module_satellite`, `module_station`, `module_access`
`app_initialOrbitDetermination`	`module_IODor`
`app_orbitDetermination`	`module_ODor`, `module_satelliteScenario`
`satelliteScenario`	`module_satelliteScenario`, `module_access`
`batch_GNSS`	`module_ODor`, `module_satellite`
`batch_position_beidou`	`module_satellite`, `module_access`
`ekf_angleData`	`module_ODor` (method=EKF)
`prediction_thruster`	`module_thruster`, `module_satellite`
`halo_orbit`	`module_halo`, `module_CRTBP`
`cartesian`	`module_cartesian`, `module_space`
`time`	`module_time`

15.7 输入文件格式

Atlas 的应用程序使用简洁的文本格式：

卫星星历文件

每行包含：`MJD x y z vx vy vz`（Cartesian, SI 单位）

观测数据文件

每行包含：`MJD 观测量1 观测量2 [...] 站编号`

场景配置文件

自定义格式的文本文件，定义卫星列表、站列表、时间范围和观测设置。

具体格式在 `release/` 目录下的 readme 文档中有详细说明。

15.8 使用建议

1. **先看 readme**：`release/` 目录下有详细的 CLI 参数说明
2. **批处理程序修改源码**：`batch_GNSS` 等批量程序的配置直接在源码中（非命令行参数），使用前需修改并重新编译
3. **场景程序**：对于复杂的多星多站仿真，推荐使用 `satelliteScenario` 而非手动编写
4. **演示程序**：`halo_orbit`，`cartesian`，`time` 是学习各模块用法的好起点

\newpage

附录 模块参考

A.1 完整模块索引（按层级）

Level 0 — 基础设施

模块	文件	功能
`module_const`	`module_const.f90`	物理常数（G, c, π, deg2rad…），地球/月球参数，单位转换因子，轨道根数类型常量，天
`module_toolKits`	`module_toolKits.f90`	通用工具：文件行计数、取整（fix）、角度归一化
`module_ErrorHandler`	`module_ErrorHandler.f90`	统一错误处理系统：错误码、错误堆栈、`RaiseEr`
`module_progressBar`	`module_progressBar.f90`	命令行进度条，支持自定义长度和前缀
`module_rotator`	`module_rotator.f90`	旋转表示转换：四元数 ↔ 欧拉角 ↔ 方向余弦矩阵
`module_randum`	`module_randum.f90`	随机数生成：均匀分布 `ran()` 和高斯分布 `norma`

模块	文件	功能
`module_String`	`module_String.f90`	字符串工具：`int2str`、字符串分割与解析
`module_FileDef`	`module_FileDef.f90`	文件类型定义、文件表管理、`OpenFile` / `CloseFil`
`module_elements`	`module_elements.f90`	轨道根数转换（第5章）
`module_MathsMatrix`	`module_MathsMatrix.f90`	基础矩阵运算：单位阵 `eye`、矩阵乘法、求逆、
`module_SP3Kit`	`module_SP3Kit.f90`	SP3 精密轨道文件格式解析（源码可能缺失）

Level 1 — 基础动力学与时间

模块	文件	功能	章节
`module_CRTBP`	`module_CRTBP.f90`	圆型限制性三体问题系统定义	第12章
`module_lambert`	`module_lambert.f90`	Lambert 问题求解器	第12章
`module_time`	`module_time.f90`	时间系统：`type_t`、MJD、时间转换	第3章
`module_EOPParam`	`module_EOPParam.f90`	地球定向参数管理	第3章

Level 2 — 行星历表与特殊轨道

模块	文件	功能
`module_matrix`	`module_matrix.f90`	高级矩阵运算：逆矩阵、线性方程组求解
`module_planets`	`module_planets.f90`	行星历表接口：JPL DE440/解析/CRTBP
`module_halo`	`module_halo.f90`	三阶解析 Halo 轨道
`module_LissajousOrbit`	`module_LissajousOrbit.f90`	Lissajous 轨道
`module_timeSeries`	`module_timeSeries.f90`	时间序列容器 `type_timeSeries`

Level 3 — 坐标与姿态

模块	文件	功能	章节
`module_space`	`module_space.f90`	坐标变换（ECI/ECF/地平/月心/NTH）	第4章
`module_SLE`	`module_SLE.f90`	线性方程组求解器（批处理最小二乘核心）	第11章
`module_attitude`	`module_attitude.f90`	航天器姿态建模	第14章
`module_cartesian`	`module_cartesian.f90`	笛卡尔坐标对象 `type_cartesian`	第4章

Level 4 — 力模型与积分器

模块	文件	功能
`module_Gravity`	`module_Gravity.f90`	引力场对象 `GravityFie`
`module_atmosphere`	`module_atmosphere.f90`	大气模型：椭球指数模型 NRLMSISE-00
`module_perturbationAspherical`	`module_perturbationAspherical.f90`	地球非球形摄动（GGM JGM-3）
`module_perturbationSRP`	`module_perturbationSRP.f90`	太阳光压：Cannonball 9参数
`module_perturbationThirdBody`	`module_perturbationThirdBody.f90`	第三体引力摄动（日月
`module_perturbationPN`	`module_perturbationPN.f90`	后牛顿效应（广义相对论 修正）
`module_thruster`	`module_thruster.f90`	推力器模型
`module_orbitPredictionNumerical`	`module_orbitPredictionNumerical.f90`	数值轨道预报（RKF78 力模型合成）

Level 5 — 应用层对象

模块	文件	功能	章节
`module_station`	`module_station.f90`	地面站对象 `type_station`	第9章
`module_satellite`	`module_satellite.f90`	卫星对象 `type_satellite`	第8章
`module_access`	`module_access.f90`	可见性分析与观测数据管理	第10章
`module_residual`	`module_residual.f90`	观测残差计算	第11章
`module_satelliteScenario`	`module_satelliteScenario.f90`	多星多站场景容器	第8章
`module_IODor`	`module_IODor.f90`	初轨确定引擎	第11章
`module_ODor`	`module_ODor.f90`	精密定轨引擎（批处理/EKF/序贯）	第11章

Level 6 — 对外接口

模块	文件	功能
`module_Atlas`	`module_Atlas.f90`	主模块：重导出 `module_ODor`，`module_IODor`，`module_sp3`

独立模块

模块	文件	功能
`module_STM`	`module_STM.f90`	解析状态转移矩阵（J2+第三体+SRP, NSG2）

Legacy 文件

文件	功能
`PLEPH.for`	JPL 行星历表二进制文件读取/插值（Fortran 77）
`nrlmsise00_sub.for`	NRLMSISE-00 大气模型子程序库（Fortran 77）

文件	功能
`nrlmsise00_driver.for`	NRLMSISE-00 测试驱动程序

A.2 关键 Derived Type 速查表

时间与坐标

类型	模块	关键字段	核
`type_t`	`module_time`	`ttype, day, sec, val`	`t%init(MJD)` 或 `t%init(y,m,d,h`
`type_cartesian`	`module_cartesian`	`position(3), velocity(3), reference, t`	`cart%init(pos,`

轨道与动力学

类型	模块	关键字段	关
`type_CRTBP`	`module_CRTBP`	`m1, m2, a12, mu, TU, L1–L5(2)`	`init`, `getLagra`
`type_halo`	`module_halo`	`c2,c3,c4, omega0, k`	`init`, `c`
`type_LissajousOrbit`	`module_LissajousOrbit`	`A_xy, A_z, phi_xy, phi_z`	`init`, `c`
`GravityField`	`module_Gravity`	`GM, referenceRadius, C(:,:), S(:,:)`	`init`, `a` `show`

对象层

类型	模块	关键字段
`type_satellite`	`module_satellite`	`states, opMod, eleType, trajectory(:), propagator`

类型	模块	关键字段
`type_opMod`	`module_satellite`	`Cd, amrDrag, F107, Ap, reflectivity, amrSRP, order, K(8`
`type_station`	`module_station`	`longitude, latitude, altitude, position(3)`
`type_thruster`	`module_thruster`	比冲、推力范围、方向、燃耗
`type_timeSeries`	`module_timeSeries`	`startTime, endTime, interval, nPoints`

定轨

类型	模块	关键字段	关键方法
`type_IODor`	`module_IODor`	(IOD 配置)	`init`, `solve`, `show`, `saveIODor`
`type_ODor`	`module_ODor`	`method` (1=批处理, 2=EKF, 3=序贯), `varGroup`	`init`, `solve`, `invoke`, `saveEph`
`type_SLE`	`module_SLE`	`BTB, BTY, X, P`	`initialize`, `addData`, `solve`, `propagate`, `dataFusion`
`type_residual`	`module_residual`	(残差数据)	—

A.3 常数速查表

物理常数（`module_const`）

常量名	值	说明
`PI`	3.141592653589793	圆周率
`deg2rad`	π/180	度→弧度转换因子
`rad2deg`	180/π	弧度→度转换因子
`SPEED_OF_LIGHT`	299792458.0	真空光速 (m/s)
`EARTH_J2_CONSTANT`	1.0826267×10^{-3}	地球 J2 项

地球参数（`module_const`）

常量名	值	说明
`LENGTH_UNIT`	6378.1363	长度单位 (km) = 1 地球赤道半径
`TIME_UNIT`	806.811634	时间单位 (s)
`MASS_UNIT`	GM_earth / G	质量单位 (kg) = 1 地球质量

单位转换因子（`module_const`）

因子	公式	用途
`u_km`	= `LENGTH_UNIT`	归一化长度 → km
`u_sec`	= `TIME_UNIT`	归一化时间 → 秒
`u_day`	= `TIME_UNIT / 86400`	归一化时间 → 天
`acc_unit`	= `LENGTH_UNIT*1000 / TIME_UNIT²`	归一化加速度 → m/s²

元素类型常量（`module_const`）

常量名	值	说明
`STATE_TYPE_KEPLERIAN`	1	开普勒根数
`STATE_TYPE_CARTESIAN`	2	笛卡尔根数
`STATE_TYPE_NONSINGULAR_1`	3	第一类非奇异根数 (NSG1)
`STATE_TYPE_NONSINGULAR_2`	4	第二类非奇异根数 (NSG2)

天体类型常量（`module_const`）

常量名	值	说明
`OBJECT_TYPE_STATION`	101	地面站
`OBJECT_TYPE_SATELLITE`	102	卫星
`OBJECT_TYPE_DRONE`	103	无人机
`OBJECT_TYPE_SHIP`	104	舰船
`OBJECT_TYPE_PLANE`	105	飞机
`OBJECT_TYPE_ROCKET`	106	火箭

A.4 力模型开关速查

索引	参数名	默认值	功能
K(1)	`k1`	1	田谐项摄动（非球形引力）
K(2)	`k2`	1	月球引力摄动
K(3)	`k3`	0	月球潮汐形变
K(4)	`k4`	1	太阳引力摄动

索引	参数名	默认值	功能
K(5)	`k5`	0	太阳潮汐形变
K(6)	`k6`	0	太阳光压 SRP
K(7)	`k7`	0	大气阻力
K(8)	`k8`	0	后牛顿效应

常用配置

场景	K(1:8)	说明
二体运动	`(0,0,0,0,0,0,0,0)`	纯 Kepler 运动
地球引力场	`(1,0,0,0,0,0,0,0)`	仅非球形引力
日月引力	`(1,1,0,1,0,0,0,0)`	默认配置（非球形+日月）
LEO 全部摄动	`(1,1,0,1,0,1,1,0)`	低轨卫星完整力模型
GNSS 精密	`(1,1,1,1,1,1,1,0)`	GNSS 卫星高精度模型
相对论 LEO	`(1,1,0,1,0,0,0,1)`	低轨卫星含 1PN 修正

A.5 SP3 文件格式

`module_SP3Kit`（Level 0）和 `module_sp3` 提供 SP3 精密轨道文件的读写支持。

SP3（Standard Product 3）是一种标准化的卫星轨道文件格式，由 IGS（International GNSS Service）等机构发布，包含：

- **头信息**：历元、卫星列表、坐标系统
- **数据行**：每隔 15 分钟（或 5 分钟）的卫星位置 (x, y, z, km) 和钟差 (μs)
- **精度信息**：位置标准差

Atlas 的 SP3 支持用于 GNSS 精密定轨（`batch_GNSS`，`batch_position_beidou`）中的轨道输

入。

A.6 数学工具速查

矩阵运算（`module_MathsMatrix`）

函数/子程序	说明
`eye(n, A)`	生成 n×n 单位矩阵
`matrix_multiply(A, B, C)`	矩阵乘法
`matrix_inverse(n, A, Ainv)`	矩阵求逆

高级矩阵（`module_matrix`）

函数/子程序	说明
`InvMatrix_LUsolver(n, A, Ainv)`	LU 分解求逆
(线性方程组求解)	—

三维向量（`module_toolKits`）

函数	说明
`cross_3d(a, b, c)`	三维向量叉乘：c = a × b

旋转矩阵（`module_rotator` / `module_space`）

子程序	说明
`Rxmatrix(theta, R)`	绕 x 轴旋转 θ
`Rymatrix(theta, R)`	绕 y 轴旋转 θ
`Rzmatrix(theta, R)`	绕 z 轴旋转 θ

子程序	说明
`coordinate_reverse(axis, R)`	坐标反向

A.7 观测类型速查

常量	值	观测量	维数
`ALPHA_DELTA`	11	(α, δ)	2
`RADAR`	12	(ρ, α, δ)	3
`RANGING`	13	(ρ)	1
`GNSS`	14	(x,y,z,vx,vy,vz)	6
`POSITION`	15	(x, y, z)	3
`AZIMUTH_ELEVATION`	16	(A, E)	2
`RANGE_DOPPLER`	17	(ρ, ρ̇)	2

A.8 外部数据文件速查

文件/路径	用途	使用模块
`data/data_op/GGM03C.txt`	地球重力场系数 (360阶)	`module_Gravity`
`data/data_op/Nutation.dat`	章动系数（20项主章动）	`module_space`
`data/data_op/EOP*`	地球定向参数表	`module_EOPParam`
`data/data_op/JPL DE440 历表*`	行星/日月历表	`PLEPH.for`, `module_planets`
`data/data_op/nrlmsise00_cheng/`	大气密度查表（按Ap/	`module_atmosphere`

文件/路径	用途	使用模块
	F107分类)	
`data/input/*`	场景配置和观测输入	各应用程序
`data/output/*`	轨迹/观测/定轨输出	各应用程序
`sofa/`	IAU SOFA 源码（编译为 `build/libsofa.a` 并随库链接)	`module_space` (IAU2006)